sensors

MDPI

Review

A Systematic Review on the Applications of UPPAAL

Iwona Grobelna [1,*], Krystian Gajewski [1] and Andrei Karatkevich [2]

[1] Institute of Automatic Control, Electronics and Electrical Engineering, University of Zielona Góra, 65-516 Zielona Góra, Poland
[2] Department of Applied Computer Science, AGH University of Science and Technology, 30-059 Kraków, Poland
* Correspondence: i.grobelna@iee.uz.zgora.pl

Abstract: This paper presents a systematic review on possible applications of the UPPAAL tool. This tool, an integrated environment for the modeling, validation, and verification of real-time systems modeled as networks of timed automata, is currently used in various domains of science and engineering. A systematic review of the literature from the years 2022 and 2023 was conducted following the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) procedure. The aim was to identify the current application areas of various versions of the UPPAAL tool, including CORA, TIGA, SMC, and Stratego. A total of 188 studies were included in the review. Quantitative information on the distribution of research papers regarding access options, scientific databases, types of papers, and geographical location was obtained. This review highlights the need for further development of the UPPAAL tool. In addition, it includes a brief comparison with other mainstream formal validation tools, explores the applicability of different UPPAAL versions, and offers practical guidelines for version selection. Finally, key open challenges and their potential solutions are discussed to support future research and tool enhancement.

Keywords: control systems; formal verification; model checking; modeling and verification; Uppaal

Academic Editors: Zheng Chen and Hao Shen

Received: 26 March 2025
Revised: 20 May 2025
Accepted: 29 May 2025
Published: 31 May 2025

Citation: Grobelna, I.; Gajewski, K.; Karatkevich, A. A Systematic Review on the Applications of UPPAAL. *Sensors* **2025**, *25*, 3484. https://doi.org/10.3390/s25113484

1. Introduction

Formal verification has been gaining popularity for decades [1], both in the scientific world and in industry. Mathematical models are analyzed and verified, gaining the most benefit in the early stages of system development. One of the most efficient methods is model checking [2]—an automatic technique for the verification of reactive systems against user-defined requirements, usually expressed as temporal logic formulas. In symbolic model checking, it can be guaranteed can that the system considered satisfies the specified requirements [3]. Otherwise, appropriate counterexamples are generated that include traces leading to undesired situations, simplifying error finding. In turn, statistical model checking [4] combines simulation and statistical methods to gain statistically valid results. It enables the prediction of system behavior with high confidence. There are numerous model checking tools available, including NuSMV [5] and its successor nuXmv [6], SPIN [7], PRISM [8], and UPPAAL [9], but also some less popular ones like HyTech [10], Ymer [11], and Zing [12]. There are also very diverse application areas that benefit from using available model checkers, from power systems [13] to Industry 5.0 with Digital Twins [14] or IoT [15] and fusion with artificial intelligence [16]. In this review article, we focus on the UPPAAL tool and examine its impact on recent and ongoing research. Formal modeling and

verification are popular emerging topics. A very recently published survey [17] identified four key research questions focusing on tool characteristics, modeling methods, verification techniques, and application domains. Our review complements that work by providing a wider perspective, expanding the scope by including a larger set of publications and obtaining some new interesting results on the application of the UPPAAL tool.

The UPPAAL tool [9] is a widely known environment for the verification, modeling, and validation of real-time systems. It is maintained and developed by two scientific institutions: Uppsaala University, Sweden, and Aalborg University, Denmark. Due to its wide range of capabilities, UPPAAL has dedicated branches for different usage. For example, UPPAAL SMC [18] is used for statistical model checking, UPPAAL Stratego [19] is used to analyze strategies, and UPPAAL TIGA [20] is used to solve games. The application areas of UPPAAL are as diverse as its versions.

The purpose of this study was to investigate the current application of the UPPAAL tool in various fields of science and to find some general statistics on its usage. A systematic review was performed following the PRISMA procedure. The following widely known scientific databases were searched: IEEE Xplore, Elsevier, Springer, ACM, MDPI, and Google Scholar. Of the 1040 primary papers found, we identified 188 suitable works following the PRISMA procedure [21] and the specified inclusion/exclusion criteria. Five research questions were defined to obtain qualitative and quantitative results. By responding to these questions, our aim was to provide summaries and insights into possible further developments of the UPPAAL tool.

The contributions of this paper are as follows:

1. Presenting a systematic review of recent research works using UPPAAL;
2. Obtaining some quantitative information on the distribution of research papers regarding access options, scientific databases, types of papers, and geographical location;
3. Analyzing the applicability and capabilities of different UPPAAL versions supported by demonstrative case studies;
4. Proposing practical guidelines for selecting the appropriate UPPAAL version based on the application context;
5. Identifying the current challenges and outlining potential future research directions and tool enhancements.

The remainder of the paper is structured as follows. Section 2 provides some background information on the UPPAAL tool, especially focusing on the different versions. Section 3 describes the research methodology and defines the main research questions. Section 4 presents the obtained results, considering both the specified research questions and some statistics. Section 5 offers an extended discussion, including a brief comparison with other mainstream formal validation tools, an analysis of the applicability of various UPPAAL versions, practical guidelines for version selection, and the identification of open challenges and their possible solutions. Finally, Section 6 concludes the article, indicates future perspectives, and lists the limitations of this study.

2. Background

Let us briefly summarize the modeling and verification tool UPPAAL and its versions. UPPAAL, first released in 1995, is an "integrated tool environment for modeling, validation, and verification of real-time systems modeled as networks of timed automata, extended with data types" (according to the home page of UPPAAL, https://uppaal.org/, last accessed 12 February 2025). It has been shaped by the need of industry for model-based validation, performance evaluation, and synthesis [22]. It is free for academic use; any other use requires a license. The UPPAAL models are specified in the form of hybrid timed automata, connected in networks, and extended with clock and data variables. The main

functions, i.e., simulation and model checking, facilitate checking model behavior. Simulation allows for the viewing of possible dynamic executions of a designed system. In turn, model checking evaluates the exhaustive dynamic behavior of a system. Reachability properties can be verified by exploring the generated state-space of a system.

In order to meet various requirements arising from the diverse application areas, UPPAAL has been extended with various features. To comply with different application areas, several versions of UPPAAL have been released.

The most well-known version is UPPAAL SMC. Since 2012, UPPAAL has been extended with statistical model checking [18] to statistically predict valid results regarding system behavior. This kind of verification has great potential, as it allows one to directly evaluate various system models under the same (often stochastic) conditions. Hybrid timed automata models can model deterministic behavior (based on states), non-linear behavior (based on ordinary differential equations), and stochastic behavior, all of them in the same models. Statistical model checking involves several runs of the system with respect to the defined properties. The results obtained are statistical, just to obtain an overall estimate of the design correctness.

Other versions of UPPAAL are just as powerful and valuable. UPPAAL Stratego [19] is mainly dedicated to strategy analysis. It enables user-friendly performance exploration of different strategies for stochastic timed games before adaptation in a final implementation. Technically, a game is a mathematical model consisting of several players (corresponding to processes) with independent objectives, usually competing, opposing, or even conflicting. UPPAAL TIGA [20] aims to solve games. It implements an efficient on-the-fly algorithm for solving games based on timed game automata. UPPAAL CORA is focused on cost-optimal reachability analysis. It uses linearly priced timed automata and finds optimal paths (with the lowest cost) to a state satisfying certain goal conditions. UPPAAL TRON is a testing tool for the black-box conformance testing of timed systems, suitable for embedded software.

3. Research Methodology

3.1. Information Sources and Search Strategy

In order to perform this exhaustive review, we searched for the keyword "uppaal" in the titles or abstracts of papers indexed in the scientific databases IEEE Xplore, Elsevier, Springer, ACM, MDPI, and Google Scholar. The search was conducted twice, the first time in September–October 2023 and the second time in January–February 2024, to include also the latest publications. The resulting papers were judged on the basis of abstract scanning. If this was insufficient, a full scan of the article was performed to check whether they were compatible with the inclusion and exclusion criteria.

3.2. Research Questions

The research carried out for this review was driven by investigating the main research questions (RQs):

RQ1: What are the application areas of the UPPAAL tool?
RQ2: Which version of UPPAAL is used the most?
RQ3: Which keywords appear the most often in the obtained papers?
RQ4: What does the distribution of research papers regarding access options, scientific databases, and types of publication look like?
RQ5: What does the distribution of research papers regarding geographical location look like?

In order to identify the current possibilities of the UPPAAL tool, we explored the recent literature as the main source to answer these research questions. Based on the research questions, the aim was to present the distribution of publications in terms of

application areas, UPPAAL versions, access options, scientific databases, and the main research countries. A wordcloud was planned to show the most popular keywords.

3.3. Eligibility Criteria

To select only relevant articles that pertained to the topic, some inclusion and exclusion criteria were defined.

The inclusion criteria were specified as follows:

- IC1: Papers published in 2022 and 2023.
- IC2: Research using UPPAAL as the main tool.

While criterion IC1 allowed us to narrow down the study to keep it up to date, criterion IC2 limited the articles to those that really focused on the UPPAAL tool.

The exclusion criteria were specified as follows:

- EC1: Papers not written in English.
- EC2: Review articles.
- EC3: Papers whose scope was to compare various tools.
- EC4: Papers that could not be evaluated due to very limited access.

Criterion EC1 eliminated papers that were not easily accessible (written in other languages than English). Criterion EC2 omitted review articles that, although they focused on the tool UPPAAL, did not use it to achieve specific goals. Criterion EC3 ignored articles that compared various verification tools. Criterion EC4 skipped articles to which there was very restricted access (no open access, limited support for scientific institutions).

3.4. Data Extraction, Storage and Analysis

The literature review was performed following the commonly used PRISMA guidelines for new systematic reviews that include database searches. The corresponding PRISMA flow diagram is shown in Figure 1. The initial search in six databases (IEEE Xplore, Elsevier, Springer, ACM, MDPI, Google Scholar) resulted in 1040 papers. It should be noted that some of the papers could not be evaluated because of very restricted access. After applying the inclusion and exclusion criteria, 188 papers were chosen for further detailed analysis.

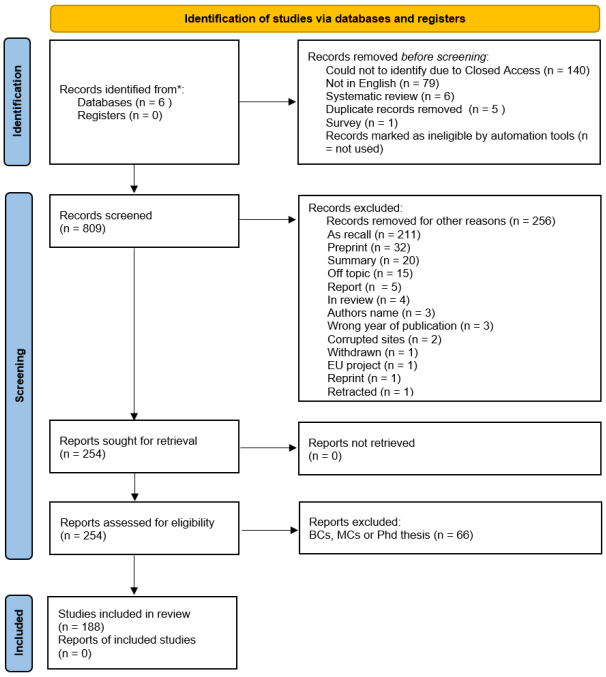

Figure 1. PRISMA flow diagram.

4. Results

This research aimed to find answers to the specified research questions. Let us discuss each of them separately.

4.1. RQ1: What Are the Application Areas of the UPPAAL *Tool?*

The distribution of publications in terms of application areas is summarized in the pie chart in Figure 2 and is illustrated in more detail in Table 1. It should be noted that not all application areas are mentioned here, only the ones with a number of publications no lower than two. The rest of the articles were classified into the group "Others".

The distribution of UPPAAL applications across various domains highlights its versatility and impact in formal verification. The largest share, 10%, belongs to verification, underscoring the foundational role of UPPAAL in model checking, system validation, and schedulability analysis. Close behind are cybersecurity and train and railway engineering (each 9%), reflecting the demand for rigorous safety and reliability standards in these critical domains. Several domains each account for 6–7%, including software, industry, communication networks, cyber–physical systems, embedded systems, and medicine. These areas benefit from modeling timing constraints, concurrency, and uncertainty—particularly in systems where correctness is vital. Power systems and robotics (each 5%) demonstrate significant use in distributed and autonomous environments. Smaller shares appear for real-time systems (4%), autonomous systems (3%), blockchain (3%), and thermal dynamics, as well as electronics and machine learning (each 2%). User journeys (1%) represents a novel application area based on modeling user–service interactions via game-theoretic approaches. The share of 7% attributed to others captures the application of the UPPAAL tool in diverse, uncategorized fields.

Let us briefly comment on specific papers and their grouping into the various application domains.

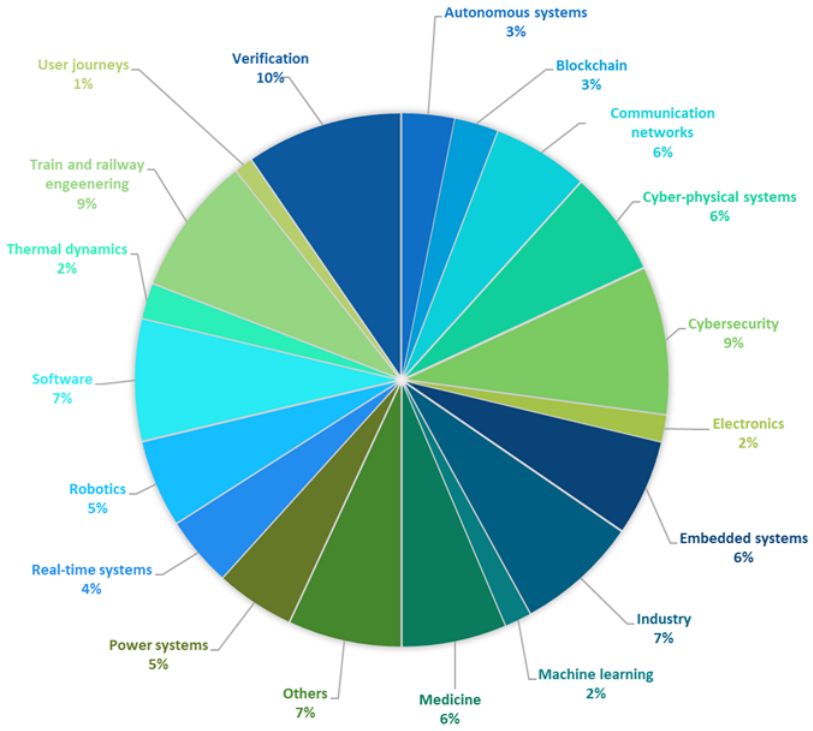

Figure 2. Most popular application areas of UPPAAL

Table 1. Most notable sectors of UPPAAL usage.

Domain	Research Papers
Autonomous systems	[23–28]
Blockchain	[29–33]
Communication networks	[34–44]
Cyber–physical systems	[45–56]
Cybersecurity	[57–73]
Electronics	[74–76]
Embedded systems	[77–87]
Industry	[88–101]
Machine learning	[102–104]
Medicine	[105–116]
Power systems	[117–125]
Real-time systems	[126–133]
Robotics	[134–143]
Software	[144–157]
Thermal dynamics	[158–161]
Train and railway engineering	[162–177]
User journeys	[178,179]
Verification	[180–197]

4.1.1. Autonomous Systems

Autonomous systems refer to the types of devices and systems that can operate and perform tasks without human intervention. This domain has been considerably expanding over the years and has become more and more relevant not only in industry but also in daily life. One of the branches of autonomous systems is the autonomous control of vehicles, both terrestrial [24–27] and aerial [28]. As they become more widespread, the ability to ensure that these systems can handle potential failures (e.g., sensor malfunctions or system breakdowns) is critical. The research papers from this area emphasize that autonomous control has to be thoroughly tested on the design level to ensure the diagnosis and exclusion of any kinds of system failures that could result, for example, in a collision. The usage of UPPAAL SMC (e.g., [28]) and Stratego (e.g., [23]) provides the ability to test the constructed formal models and verify their ability to withstand and react to different types of breakage within them and ensure their safety. In [25], the authors find that by using the UPPAAL tool, the reliability and realism of virtual testing are enhanced, improving the validity and precision of the testing results.

In the domain of autonomous systems, especially autonomous vehicles, UPPAAL offers a powerful framework for the formal verification of complex and time-sensitive behaviors. Studies highlight that formal verification with UPPAAL enhances the reliability of such systems. Moreover, it enhances the virtual testing process, making it not only more reliable but also more realistic, which ultimately leads to safer, more robust autonomous systems.

4.1.2. Blockchain

Blockchain is an emerging technology that supports peer-to-peer trade. It records transactions across many computers in a way that ensures both the security and immutability of the exchanged data. As the mentioned aspects are especially important and up-to-date, their

verification is significant. The UPPAAL tool is used for the runtime monitoring of blockchain executions [29], to evaluate system accuracy without contradictions or errors [30], to verify the framework implementation in smart contracts [31], to check the correctness of smart contracts [32], or for the compliance checking of cloud providers [33].

In the rapidly evolving domain of blockchain technology, ensuring the security and integrity of transactions, smart contracts, and compliance frameworks is essential. The UPPAAL tool plays a vital role in verifying these aspects by enabling formal verification for runtime monitoring, system accuracy, smart contract correctness, and cloud compliance. By applying UPPAAL to blockchain systems, developers can ensure that blockchain networks and smart contracts are robust, reliable, and secure, which is essential for enabling their widespread adoption and ensuring their trustworthiness in various industries.

4.1.3. Communication Networks

Communication networks enable the exchange and flow of data between individuals. They consist of nodes that communicate with each other. Their existence in daily life enables efficient information flow, especially with fast data transmission. In this area, UPPAAL is applied to check deadlock freedom and find worst-case message delivery times for message flows [34]. The Internet of Things (IoT) is a special type of communication network that connects physical devices and other objects with embedded sensors and has recently gained popularity in many applications. The temporal properties of such networks can be verified [35]. Authors have assessed UPPAAL as a tool with an intuitive and understandable graphical representation.A self-adaptive IoT system is modeled and verified in [36], in a case study of a smart home with fire detection and an automated lighting system. A Sigfox module for Network Simulator 3 is evaluated in [37]. Another study models and verifies a Sigfox-based IoT network with UPPAAL SMC [38]. The lifetime of the nodes is analyzed as a performance metric. Moreover, a set of strategies is evaluated to optimize the battery lifetime of the nodes. An IoT network may be infected with malicious software and then controlled remotely (such an infected network is usually referred to as a botnet). In [39], the dynamic behavior of a Mirai botnet, its infrastructure (used in DDoS attacks), and various categories of IoT devices are modeled and simulated. The possibility of restarting is evaluated as a defense strategy against botnets. The security of IoT networks is also verified in [40]. 5G, as the fifth generation of wireless technology, is also a valid research object. In [41], dynamic service orchestration is modeled and verified. The developed supporting tool does not require any prior experience with timed automata. In [42], a framework is proposed to analyze the RAP (Random-Access Procedure) network protocol with UPPAAL and statistical model checking. The Precision Time Protocol for the clock synchronization algorithm in automotive Ethernet is formally modeled in [43]. A new TCP protocol is proposed in [44], with modeling and simulation in UPPAAL SMC.

UPPAAL has been extensively applied to the modeling and verification of communication networks, including general message flow analysis, deadlock detection, and worst-case delivery time estimation. It is used in IoT systems to verify temporal and security properties, as well as to evaluate some performance metrics, such as node lifetime and energy efficiency. Moreover, the considered studies highlight the versatility of UPPAAL in addressing critical aspects of communication networks, particularly in the context of smart home automation, 5G protocols, dynamic service orchestration, or defense strategies against IoT botnets. Usage of the tool can lead to the development of more efficient communication systems, especially in complex, real-time environments such as industrial networks.

4.1.4. Cyber–Physical Systems

Cyber–physical systems (CPSs) integrate computation with physical processes. They are considered to be the core of Industry 4.0. As they usually involve several different technologies, they require more attention than standard software or hardware projects. So, a framework for modeling and analyzing CPSs with the application of SMC is proposed in [45]. SysML is used as a primary specification, while Enhanced Activity Calculus is used for the construction of equivalent-priced timed automata models. Another framework for the design of resilient CPSs with control theory is introduced in [47]. It helps to ensure system stability and safety. A library for the analysis and synthesis of the sampling behavior of event-triggered control systems is presented in [53], with UPPAAL Stratego employed for the synthesis of schedulers. Another study, [54], specifies a methodology for the development and security of CPSs. The software in a distributed CPS is formally verified in [48], focusing especially on timing analysis. An intelligent mechatronic component is simulated and validated in [49]. Similar work on the co-simulation of a complex CPS is presented in [50]. UPPAAL SMC is used for the validation of strategy switching to improve the fault tolerance of resource-constrained real-time applications [51].

An interesting approach is to use timed games and UPPAAL TIGA to determine when an update to a CPS is possible at certain run-time [46]. Up-to-date practical research deals with the Digital Twin (model replica of a physical system) setup for safety-aware optimization, which is verified in UPPAAL [52]. The tool also helps identify potential threats to CPSs [55]. Cyber bio-analytical physical systems are designed in [56], with UPPAAL Stratego employed for the analysis of the interaction of several devices.

Due to the multidisciplinary nature and complex interaction patterns of cyber–physical systems, they demand rigorous design methodologies beyond those employed in conventional software or hardware systems. A diverse range of formal approaches have been proposed to support modeling, verification, and synthesis, with a strong emphasis on correctness, safety, and resilience. Applications range from secure CPS development and timing verification in distributed systems to simulation, co-simulation, and Digital Twin implementations. A notable trend involves extending CPS modeling to domains such as cyber bio-analytical systems. The considered studies reveal a common focus on early-stage verification, real-time constraints, and adaptive control under uncertainty. Most recent approaches focus on modeling, proposing either a new methodology or even a whole framework.

4.1.5. Cybersecurity

Cybersecurity usually refers to protecting systems, networks, or programs (in general, hardware and software) from digital attacks. It is constantly being updated due to increasing threats. In [57], a new formalism for the defense of moving targets is proposed, and the attack time and cost distributions are calculated under various attacker strategies. The authors use UPPAAL Stratego, although they consider implementing their own tool to find the best strategies. A user-specific security policy is generated through the formal modeling of user behavior in [58]. The authors show how to identify and select the essential characteristics that define user security behavior. The identified behaviors are modeled for the purpose of automated reasoning. This allows weaknesses to be found in users' security behavior and enables the proposal of some relevant policies.

In [59] a modeling and analysis method for industrial control system functions is proposed to ensure that supervisors can work properly under potential cyberattacks. Similar work by the same authors, this time with a resilient third-party monitoring system, is also presented in [62]. The security verification of cyber–physical systems is also addressed in [60], where the authors find that exploring the human–machine interaction requires

performing an exhaustive search for each state in all combinations of feasible models. Distributed Denial of Service (DDoS) is formally specified in [61]. The functional requirements of the protocol are verified in order to determine the accuracy of the system. The attack resistance of a controller area network system (CAN) is enhanced in [63]. A comprehensive model combining a variable attacker with a CAN bus is proposed, and UPPAAL SMC is applied to determine the statistical probability of transmission and response behavior of the CAN bus. An effective property-checking method and a formal verification framework for hardware Trojan detection are proposed in [64]. The model-based risk analysis framework of the Attack–Fault Maintenance Tree is verified in [65], with some statistically valid safety/security metrics, and the impact of coordinated cyber–physical attacks is analyzed in [66]. The authors state that statistical model checking allows better scaling of the system dimension under security analysis. Moreover, the modeling language provided by UPPAAL SMC has proven to be effective in specifying realistic systems.

A three-side-channel attack is analyzed in [69], with probabilistic hyper-property logic for stochastic hybrid and timed systems. Safety violations are effectively detected through randomized reachability analysis in [67]. The work is published with the affiliation of Aalborg University, Denmark, and additionally provides some implementation details regarding the UPPAAL tool (with a reduction in the checking time of some properties from 23 h to 23 s). An UPPAAL execution engine and UPPAAL TRON were used for the development of an MUPPAAL tool [68] that allows mutation testing (artificial faults are injected into the system and the ability of tests to distinguish these mutants is evaluated). A SIM box fraud prevention system utilizing fingerprint-based access policies is analyzed in [70], with a detailed security analysis of the access control list. An ontology-based framework for formal verification of the safety and security properties of control logic is introduced in [71]. Model-driven software development with the analysis of safety and security properties is discussed in [72], where the authors point out that the time-consuming process of model checking may disqualify this technique among programmers. Voting protocols are the subject of [73], where the authors try to make security measurable.

UPPAAL and its extensions have been successfully applied in the field of cybersecurity to model, simulate, and verify a wide range of security-critical systems. These include the formal analysis of attack–defense strategies, the generation of user-specific security policies, and the verification of system resilience under cyberattacks in industrial control systems. Specific applications include the modeling of DDoS attacks, the enhancement of CAN bus security, the and verification of hardware Trojan detection mechanisms. Statistical model checking has proven particularly useful for evaluating system behavior under uncertainty and scaling to large system dimensions. Further contributions include the analysis of side-channel attacks, randomized reachability for safety violation detection, and mutation testing for software robustness. Researchers have also explored formal frameworks for SIM box fraud prevention, the ontology-based verification of control logic, and secure model-driven software development. While formal verification using UPPAAL offers strong guarantees, some studies highlight the trade-off in verification time, which may hinder its widespread adoption among developers.

4.1.6. Electronics

This narrow specialization also benefits from the UPPAAL tool. In [74], the logic of the processor's behavior-level code is analyzed. Among others, deadlock freedom is confirmed. In [75], an embedded pulse-transfer-level language for superconductor electronics is proposed, together with a framework, and verified with UPPAAL. Multicore processors and interference analysis are the subject of [76], where UPPAAL is used to

compute an upper bound of the number of interferences experienced by each task in each component for each segment.

In the domain of electronics, simulation-based verification remains dominant due to its scalability and widespread tool support, while formal verification, including model checking, is used more selectively for verifying complex control logic, timing correctness, and safety-critical properties in electronics. The considered studies confirm that they benefit from using UPPAAL through its capability to model and verify low-level hardware behavior and timing-critical systems.

4.1.7. Embedded Systems

Embedded systems combine hardware and software and are designed for a specific function. The feasibility of using ciphers in an embedded real-time operating system (RTOS) is investigated in [77]. An operating system architecture for sustainable embedded systems is proposed in [78], where application tasks are modeled with UPPAAL (incorporated into networks) and different aspects of (non)functional properties are verified. In [79], a modeling concept for the formal verification of compositional software based on an operating system is introduced, with an RTOS kernel used as the modular model, and verified in terms of task synchronization and resource management timing. In [80], a time-related model checking approach is proposed for the specification of software requirements in embedded systems to show possible software behaviors. The safety-critical embedded systems of avionics are evaluated in [81].

A middleware that supports the development of embedded multi-agent systems to prevent a lack of connectivity is presented in [82], with formal modeling and verification performed using the UPPAAL tool. Various communication methods are evaluated. Model checking is combined with reinforcement learning to solve the multi-agent autonomous system planning problem in [83]. A new method called MoCRel is integrated into UPPAAL Stratego, and the experiments show that it can solve the planning problem in complex maps with large numbers of agents performing various types of tasks. Similar work on path planning and task scheduling strategies for multiple autonomous agents [84] resulted in the integration of the improved MCRL method into UPPAAL Stratego. An intuitive agent-based abstraction scheme is studied in [85], with a reduction in state space achieved. The correctness of the approach is formally proven. A multi-agent reasoning-based context-aware model is proposed in [86], with formal verification of the correctness properties achieved. The liveness and real-time requirements of OS-based embedded software are analyzed in [87]. The limited number of properties is considered; nevertheless, the proposed modeling strategy is said to be scalable.

The analyzed studies show that UPPAAL has been widely utilized in the formal verification of embedded systems, enabling the analysis of real-time properties, task scheduling, and resource management in RTOS-based architectures. Its application is extended to safety-critical embedded systems and multi-agent systems, where integration with methods like MoCRel and MCRL (for UPPAAL Stratego) facilitates scalable planning and verification. These studies demonstrate the effectiveness of the tool in ensuring correctness and good performance in complex, time-sensitive embedded environments.

4.1.8. Industry

All pure industrial approaches have been included in this category. An industrial control network protocol is modeled and its reliability is verified in [88]. Collaborative manufacturing is modeled and the production cost is simulated in [89]. An intelligent product line manufacturing system is verified in [90]. In [91], a twin-based digital automatic programming method is introduced for the adaptive control of manufacturing cells using the

simulation feature of UPPAAL. A complete process for assessing the robustness of schedule solutions is proposed in [93], supported by UPPAAL SMC. A reconfigurable fault-tolerant control framework applied to a manufacturing system is presented in [94], and is verified before implementation with UPPAAL. The safety of multiple industrial robot manipulators with path conflicts is verified in [92]. Verification of the human-adapted programmable logic controller (PLC) code, according to the IEC 61131-3 standard, is discussed in [95]. An ontology-based framework for industrial control systems is introduced in [96]. A similar automated tool-supported quantitative risk analysis framework is also proposed in [97], using UPPAAL SMC for simulations. A Brake-by-Wire industrial prototype system that improves road safety is used as a case study in [98]. An industrial manufacturing control system is extended with system defenses in [99], providing some recommendations for the development of future defensive strategies. The construction industry is the research area of focus in [100], where the authors conduct formal modeling and verification of the credibility of knowledge. An analysis of the scalability and performance of a persistent storage approach is performed in [101], where fault tolerance and data consistency are verified with UPPAAL.

In industrial systems, UPPAAL has been extensively applied to model, simulate, and verify control logic, system robustness, robotic manipulators, and real-time constraints across various manufacturing and automation scenarios. Its applications include the verification of industrial protocols, cost-aware scheduling, fault-tolerant control frameworks, and collaborative manufacturing systems. An interesting approach is the validation of human-adapted PLC code in compliance with IEC standards. Case studies such as Brake-by-Wire systems and persistent storage solutions demonstrate the capacity of UPPAAL to analyze safety, scalability, and fault tolerance. These contributions underscore that UPPAAL is valuable in enhancing reliability and security in complex industrial automation environments.

4.1.9. Machine Learning

Machine learning is considered a type of artificial intelligence (AI) in which computers learn from the provided data and improve with experience. In [102], the partitioning-refinement learning method of UPPAAL Stratego reduces the expected number of guesses in the popular Wordle game by almost half. In [103], a technique is proposed to learn explainable timed automata from passive observations of a black-box function. A black-box function may be an artificial intelligence system. A prototype was implemented and evaluated by learning two controllers, namely a brick-sorting conveyor belt trained with reinforcement learning and a real-world derived smart traffic light controller. A novel approach to analyze large adaptation spaces is proposed in [104]. Using classic supervised machine learning techniques to reduce adaptation spaces on the fly is suggested. The analysis of the adaptation options is performed with UPPAAL SMC.

UPPAAL, particularly the Stratego and SMC versions, has been applied to integrate formal verification with machine learning techniques. It supports learning optimal strategies, as demonstrated in strategy synthesis for games like Wordle, and in explainable model inference from black-box systems using timed automata. Its applications include reinforcement learning-trained controllers, real-world systems such as traffic lights and sorting conveyors, and the analysis of large adaptation spaces by incorporating supervised learning to reduce complexity during system adaptation.

4.1.10. Medicine

Medicine is a branch that is very important for entire populations around the world. Technological progress offers new possibilities for better treatment of patients, but at the

same time, any new solution must be fully reliable to ensure people's safety. A medical resource utilization process is modeled in [105]. Afterwards, it is verified with UPPAAL to ensure that all safety requirements are met. A healthcare system Serums is introduced in [106], where formal methods are used to verify its safety and security requirements. The quality of service of a healthcare system is ensured with UPPAAL in [107], especially focusing on reliability and security. In a runtime environment, a clinically interpretable classification of arrhythmias is verified in [108]. The efficacy of the approach is evaluated in conjunction with existing clinical ECG databases. A healthcare application based on the Internet of Things is formally analyzed in [109], considering mainly the properties of safety, liveness, and deadlock freedom. A unified healthcare communication system is analyzed against threats in [110]. UPPAAL SMC is used to detect the most probable type of attacks resulting in the mistreatment of patients. In turn, a telerehabilitation system is formally verified in [111] that also promotes formal methods for the design of safe medical software systems.

Special attention has been paid to specific illnesses. Deep brain stimulation controllers for Parkinson's disease treatment are investigated in [112], and Alzheimer's disease is the research subject in [113], while prevention strategies for COVID-19 exposure are analyzed in [114] (using UPPAAL Stratego). The functional prototype of a mechanical ventilator is verified and improved in [115]. A bioelectronic system connecting medical devices and a biological system is verified in [116] to check properties such as the reachability of hazard-related states.

Medicine is a critical domain where technological innovations must meet the highest standards of safety and reliability. In this context, UPPAAL has been successfully employed to formally verify a wide range of healthcare applications, ensuring adherence to essential safety, security, and liveness properties. These include medical resource utilization processes, IoT-based healthcare systems, quality of service in healthcare infrastructures, and clinically relevant models such as arrhythmia classification, Alzheimer's models, or Covid-19 exposure prevention strategies. The considered studies demonstrate how formal verification contributes to the development of dependable and secure medical systems.

4.1.11. Power Systems

Power systems consist of electrical components and are used to supply and transfer electrical power. The performance of the predictive control algorithms of a Finite Set model applied to a matrix converter is statistically verified in [117,125]. The results obtained can be used to extend the lifetime of these devices during various grid conditions. Protection systems in low-voltage distribution grids are formally verified in [118,123]. Power smart IoT entity services are modeled in [119] in order to increase their feasibility and stability. A new mobility- and energy-harvesting-aware medium-access control protocol is modeled in [120]. Its performance is evaluated with UPPAAL SMC. A control strategy for a battery management system is checked with UPPAAL Stratego in [121]. A case study of a research nuclear reactor is presented in [122]. Here, UPPAAL Stratego is used to find the number of spares that minimizes the total costs of downtime and purchase of spares, all in a short period of time. The flexible behavior of energy systems to balance the production and consumption of energy is modeled and analyzed in [124].

Power systems increasingly rely on formal verification to ensure efficiency, safety, and adaptability. UPPAAL and its extensions (SMC and Stratego) have been applied to verify control strategies for converters, battery systems, and grid protection mechanisms. It also supports the modeling of energy-aware IoT services, communication protocols, and flexible

energy balancing. These applications demonstrate that UPPAAL is able to support strategic decision-making in modern energy infrastructures.

4.1.12. Real-Time Systems

Real-time systems are supposed to respond within a specific time period to incoming inputs and commands. The schedulability of an acquisition–execution–restitution task model is analyzed in [126]. Embedded real-time systems are verified in [127], checking, e.g., the bounded response. Execution strategies for temporal networks with various sources of uncertainty are computed in [128] with the modeling of possible reaction times (using UPPAAL TIGA). The time-sensitive software-defined network architecture is verified in [129], among others, against deadlock freedom and starvation freedom. A framework for the quantitative evaluation of cyber–physical–social system performance is modeled and verified with UPPAAL SMC in [130]. Equivalent mutants in real-time model-based mutation testing are detected with UPPAAL TIGA in [131]. The same version of the tool is used in [132] to synthesize safe controllers for continuous-time sampled switched systems. Event logs in real-time systems are investigated in [133] with classical UPPAAL and UPPAAL SMC.

Real-time systems require strict timing guarantees, and UPPAAL—along with its TIGA and SMC extensions—has proven to be effective for their formal verification. It is applied to analyze task schedulability, bounded response times, execution strategies under uncertainty, and time-sensitive network properties; for detecting equivalent mutants in testing; and for synthesizing safe controllers. These applications show that UPPAAL is suitable for verifying the correctness and reliability of time-critical systems.

4.1.13. Robotics

Robotics is a branch of engineering focused on the design, creation, and operation of robots that can perform their tasks automatically. In [134], multi-robot interactive scenarios in service settings are formally modeled and verified with UPPAAL SMC. Interactive robot service applications are also the topic of [135], where a model-driven framework integrated with UPPAAL SMC is presented. Real-time autonomous robots are formally verified in [136], with several important requirements checked. Dynamic route planning for a fleet of autonomous mobile robots is discussed in [137] with the application of UPPAAL Stratego. The correct behavior of multi-robot autonomous systems is ensured in [138], where a framework is introduced that integrates design and formal verification at a higher level of abstraction. The impact of human error on interactive service robotics scenarios is analyzed in [139]. Latencies and buffer overflow in distributed robotic systems are investigated in [140]. The work highlights the advantages of UPPAAL and indicates that it can reveal potential errors that are not detected by experiments. Service robots with uncertain human behavior are considered in [141], with the proposal of a framework built on formal modeling, verification, and learning techniques. Explainable service robots and their software architecture are addressed in [142], with a formal analysis conducted using UPPAAL SMC. In [143], a decentralized solution for high-level multi-agent task planning problems is proposed, with UPPAAL used to synthesize a plan that provably satisfies the updated task.

Robotics benefits significantly from formal verification using UPPAAL and its SMC and Stratego extensions, in particular, to verify multi-robot systems, autonomous navigation, and real-time service applications. Key contributions include route planning, task coordination, handling uncertain human behavior, and evaluating performance issues like latency and buffer overflows. So far, UPPAAL has been proven to enhance reliability in the domain of robotics, especially in human-interactive and real-time systems.

4.1.14. Software

Software engineering aims to create reliable, efficient, and scalable applications that we use everyday. Additional verification evidently contributes to an increase in quality. As history shows, software failures can be expensive and have catastrophic consequences, as was the case with Ariane 5. In 1996, the flight ended in failure, as 40 s after initiation, the launcher broke apart and exploded. The Inquiry Board report can be found at https://www.esa.int/Newsroom/Press_Releases/Ariane_501_-_Presentation_of_Inquiry_Board_report (last accessed on 14 February 2025). In this context, the authors of [144] validate RabbitMQ—an implementation of the Advanced Message Queuing Protocol. The basic properties are checked, for example, data reachability or data concurrency. An open-source business process management system, YAWL, is verified in [145]. An automata-based approach to manage self-adaptive component-based architecture is proposed in [146], where the consistency of the software is checked before the adaptation implementation. In [147], a new runtime environment is introduced for the coordination of services in contract-based applications. The absence of deadlock was verified with UPPAAL. The time behavior of self-adaptive software under uncertainty is modeled and analyzed in [148], and the probabilistic behaviors of the model are verified with UPPAAL SMC. Distributed shared-memory algorithms are formally checked in [149], with the authors employing problems related to state explosion.Context-oriented chatbot conversational flows are modeled in [150], allowing one to overcome some verification gaps that are not able to be overcome via other testing techniques. Colluder detection in SaaS (software as a service) cloud applications with subscription-based licenses is the subject of [151]. An integrated co-simulation and synthesis framework for the stochastic model-predictive control of software controllers (called STOMPC) is proposed in [152], with UPPAAL Stratego employed as the engine. It is said to be generally applicable across different application domains, including traffic light control or building floor heating. In [153], it is used for synthesizing safe and near-optimal control strategies for stormwater detention ponds. Self-adaptive systems are the subject of [154], where UPPAAL SMC is used to verify adaptation options. Similarly, reconfiguration strategies for self-adaptive systems are precomputed with UPPAAL Stratego in [155]. Near-optimal solutions are approximated. Another approach for engineering self-adaptive systems is proposed in [156] with the application of UPPAAL SMC. The real-time behavior of an application for tennis training is verified in [157] to ensure that the safety requirements are met.

UPPAAL plays a critical role in improving software reliability, particularly in systems where failures can have catastrophic effects. It has been used to verify messaging systems (e.g., RabbitMQ) and business process tools (YAWL). UPPAAL SMC has been successfully applied for analyzing probabilistic behaviors in adaptive systems, runtime environments, and chatbot interactions, as well as in co-simulation frameworks for traffic and building control. In turn, UPPAAL Stratego has been applied to synthesize near-optimal adaptation strategies and control policies in contexts such as SaaS license enforcement, stormwater detention systems, and self-reconfigurable software systems.

4.1.15. Thermal Dynamics

Thermal dynamics is gaining importance with the growing popularity of sustainability, eco-friendliness, and energy-saving policies. A toolchain for controlling a domestic heat pump in a floor heating system is proposed in [158,160]. The predictive model is prepared and evaluated with the use of UPPAAL Stratego. The same version of UPPAAL is applied for residential heat pumps with uncertain weather forecasts in [159]. The minimum and maximum flexibility potentials of the pumps in optimistic and pessimistic energy consumption patterns are calculated, and the impact of weather forecast on the flexibility of heat

pumps is investigated. A similar research topic, namely the thermal dynamics of residential buildings with energy flexibility, is addressed in [161], where the heat-to-power flexibility of heat pumps is evaluated.

Thermal dynamics benefits from using UPPAAL, in particular, its Stratego extension, for optimizing heat pump control in residential heating systems. Predictive models have been synthesized and evaluated under uncertain weather conditions to assess flexibility potential. Studies show how forecast variability impacts energy flexibility and the heat-to-power adaptability of domestic heat pumps.

4.1.16. Train and Railway Engineering

Train and railway engineering focuses on the development and maintenance of modern railway infrastructure. Railways are one of the most popular and energy-efficient ways to transport both humans and cargo. Moreover, in many countries they are considered to be part of critical infrastructure. Due to these facts, it is essential to provide the highest standards of safety and perform meticulous verification of all components, i.e., systems [164,167], algorithms, and communication protocols [163], before their implementation to ensure reliability and minimize the risks of accidents. In [162], a formally verified scheme is proposed to manage train communication information. The application of a Unified Modeling Language (UML) supporting railway engineers, together with UPPAAL, is shown in [165]. A risk evaluation method for autonomous trains is proposed in [166]. A description of the usage of UPPAAL for the formal verification of the ERTMS (European Rail Traffic Management System)/ECTS (European Train Control System) can be found in [168,169,171,173]. A novel approach for designing electronic urban trains via model verification is presented in [170]. A movement authority scenario in a train-centric communication-based train control system is analyzed to determine its safety in [172] with UPPAAL SMC. Urban rail transit is the topic of [174], where the authors conduct formal verification of security requirements. Models of a safety-critical motor controller in railway systems are evaluated in [175]. An interesting discussion on research into formal methods that can contribute to the development of modern railway systems is presented in [176]. Future train control systems are also considered in [177].

Train and railway engineering increasingly relies on formal verification. UPPAAL is widely applied to validate railway systems, algorithms, and communication protocols, and ensure compliance with safety standards. Its specific applications include ERTMS/ETCS verification, autonomous and urban train systems, risk evaluation, and movement authority scenarios. Model-driven approaches and UML integration further support reliable system development in this safety-critical sector.

4.1.17. User Journeys

A separate category is dedicated to user journeys. This refers to the process a user goes through when interacting with a service or a system. User journeys are formalized as weighted games (user versus service provider) in [178]. UPPAAL Stratego is used to discover challenges in the interaction between customers and a company. The other focus of their research is on multiparty event logs (an extension of event logs with information on the parties) [179] that allow the analysis of user journeys.

By treating user journeys as strategic games or enriched event logs, UPPAAL Stratego enables the precise identification of interaction challenges and behavioral insights, supporting the design of more reliable and user-centric systems.

4.1.18. Verification

In general, the key application of the UPPAAL tool is the exact verification of various kinds of systems. In this category, we have classified works that contribute considerably to

verification methods, although they could also be considered to fit into one of the other categories. In [180], a tool is proposed that aims to integrate formal analysis and the verification of functional requirements. The robustness of timed automata is analyzed in [181]. UPPAAL is used here for sufficiency checks and for computing witnesses of the proposed methods.

Stochastic time automata and proof of their correctness are the subject of [182]. The zone-based verification of timed automata is considered in [183]. Hardware/software co-design with UPPAAL SMC is addressed in [184], going from UML MARTE (providing foundations for model-based descriptions of real-time and embedded systems) specification to early functional verification. The practical aspects of test automation with efficient test models developed by the authors are considered in [185], with the aim of reducing the effort required to create models. Compliance through model checking is addressed in [186]. Model-based testing is combined with automated analysis in [187], focusing especially on reachability and deadlock freedom properties. An approach to constructing a target clock state in a model with sequences of difference-bound matrix operations is proposed in [188]. In [189], a learning-based framework is introduced for assume/guarantee reasoning.

The use of Monte Carlo Tree Search for model checking is evaluated in [190], with the experiments performed in UPPAAL CORA. A rare event simulation technique is incorporated into UPPAAL SMC in [191]. A property specification pattern catalog is proposed in [192], allowing practitioners to specify qualitative requirements based on patterns (eliminating the use of temporal logic). Aspect-oriented modeling, where correctness is ensured by the construction, is presented in [193]. Temporal modalities to extend the notion of assume/guarantee contracts are introduced in [194], focusing on practical aspects of test automation. The diverse aspects of modeling and quality assessment are discussed in [195]. Dynamic timed automata for the modeling and verification of reconfigurable systems are proposed in [196], and are transformed into semantic equivalent timed automata in UPPAAL format. An open-source tool, Uppex, is described in [197], which automatizes feature analysis by combining Microsoft Excel spreadsheets and UPPAAL models. This allows the authors to reach the right balance in the level of details—enough detail to be trustworthy but not so much that it hinders the verification of complex requirements.

The UPPAAL tool plays a central role in the formal verification of diverse system models, with numerous studies advancing its methodologies beyond domain-specific applications. Key theoretical developments include the robustness analysis of timed and stochastic automata, zone-based verification techniques, and assume/guarantee reasoning frameworks. Complementary to these are practical contributions aimed at improving the efficiency and accessibility of verification, such as model-based testing, compliance verification, and automated test generation. The considered studies demonstrate the rich versatility of UPPAAL and its sustained relevance in advancing both the theory and practice of system verification.

4.1.19. Others

In this category, we have placed articles that do not strictly match the particular application areas considered above. Nevertheless, it should be noted that this is only our subjective opinion, and some aspects relevant to different domains can also be identified in these papers.

Stochastic Reward Nets (a subtype of Petri nets) are formally reduced and analyzed in [198]. A novel modeling and analysis approach is proposed, aimed at checking model correctness. UML state machines are translated into timed automata in [199] using UPPAAL semantics. Twin clutch gear control to support drivers is formally verified in [200], and the results indicate that the model partially meets its functional requirements. The application

of UPPAAL SMC to comply with the safety and efficiency control laws of multi-car elevator systems is investigated in [201].

In [202], a gossip-based information dissemination protocol is introduced to improve distributed system resiliency, where client–server systems are modeled and analyzed with UPPAAL SMC. Similar work by the same authors [203] evaluates the efficacy of the proposed approaches in improving the relative performance of three models. Asynchronous systems with a timed integrated model of distributed systems are successfully modeled and verified in [204] for small, medium-sized, and large system models. Digital Twins are addressed in [205], where their foundation model is formally verified in UPPAAL. A methodology and tool, called A2A, that automatically models systems defined by the Autosar specifications as timed automata is proposed in [206]. The timing properties of the model are then verified using UPPAAL. The clock synchronization algorithm of an in-vehicle network, FlexRay, is formally modeled and verified in [207]. An approach to building better trust in human–machine teaming by combining model checking and machine learning is presented in [208] and verified with UPPAAL SMC.

The behavior of an unmanned aerial vehicle (UAV) cluster is modeled in [209], and the authors perform formal verification of a cluster attack mission. In [210], an assume/guarantee framework for additive compositional reasoning in the setting of hybrid systems is presented. The authors show how UPPAAL SMC may be used to efficiently falsify refinements.

This category encompasses diverse applications of the UPPAAL tool not classified under the other domains listed above, demonstrating the versatility of UPPAAL in modeling and verification. Several works explore foundational modeling transformations, such as translating UML state machines into timed automata or reducing Stochastic Reward Nets to verify correctness. UPPAAL's Applications range from verifying control logic in automotive systems to improving the safety and efficiency of digital infrastructure. Notably, UPPAAL SMC is employed in research focused on large-scale asynchronous and distributed systems, showcasing its scalability and robustness. Emerging domains like Digital Twins and UAV clusters also benefit from formal verification. These contributions highlight the adaptability of UPPAAL across novel and complex system architectures, reinforcing its role as a versatile tool in formal verification research.

4.2. RQ2: Which Version of UPPAAL *Is Used the Most?*

The distribution of publications in terms of the UPPAAL version is summarized in the graph in Figure 3. It should be noted that the results are based on the analysis of papers that explicitly reported the tool version (n = 76). As can be seen in the diagram, UPPAAL SMC dominates the landscape, accounting for 58% of publications. This prevalence underscores its growing relevance in contemporary system design, particularly for applications requiring stochastic modeling, performance evaluation, and probabilistic verification. It also indicates an important future research direction. UPPAAL Stratego, used in 29% of the studies, reflects the increasing interest in synthesis and strategy optimization under uncertainty. UPPAAL TIGA accounts for 12% of the total share, highlighting its niche application in controller synthesis within timed games. UPPAAL CORA is used occasionally (referenced in only 1% of the papers), indicating limited but focused usage, likely due to its specialization in cost-optimal reachability analysis. The prominence of the SMC and Stratego extensions suggests a research shift toward quantitative analysis and automated strategy generation, aligning with trends in cyber–physical systems and adaptive control.

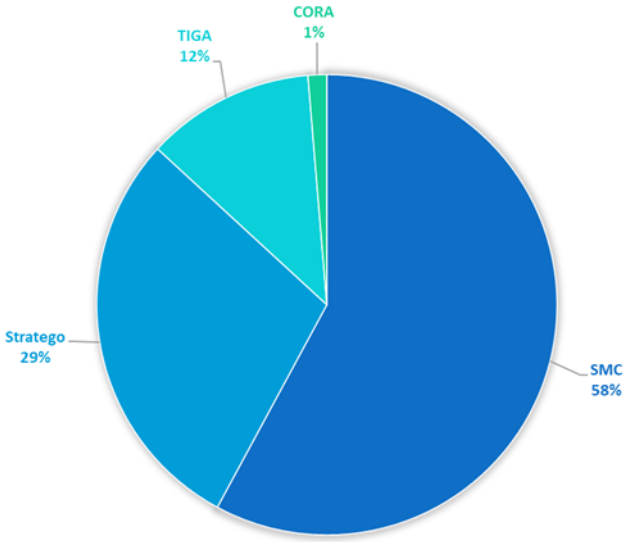

Figure 3. UPPAAL versions (explicitly mentioned in 76 papers).

4.3. RQ3: Which Keywords Appear the Most Often in the Obtained Papers?

During this study, we collected the keywords that occurred in the papers that used UPPAAL. Their frequency of use is illustrated in Figure 4, where the larger the font, the more often a given word appeared. As expected, "model checking" emerges as the most prominent term, emphasizing the central role of UPPAAL in formal verification processes. Other frequently occurring terms include "timed automata" and "formal verification", which highlight the tool's foundational basis in timed models and its application in rigorous system correctness analysis. The presence of terms such as "cyber-physical systems", "safety", and "real-time systems" further reflects the widespread use of UPPAAL in verifying critical timing and reliability requirements in modern system design. The keyword distribution confirms strong alignment of UPPAAL with contemporary challenges in verifying complex, time-sensitive, and safety-critical systems.

Additionally, we prepared a keyword co-occurrence network visualization, shown in Figure 5. Each node represents a keyword, while the edges indicate co-occurrence links across the analyzed publications. The size of the nodes, similarly to in a wordcloud, reflects the frequency of keyword appearances, and their proximity and clustering suggest thematic affinities. The largest and most central nodes—"model checking", "timed automata", and "statistical model checking"—form the core of the network, reaffirming their foundational role. Surrounding these central terms are several interconnected clusters, each representing a distinct thematic focus. For instance, the dark green cluster includes keywords such as "machine learning", "planning", and "behavioral sciences", reflecting the growing interest in integrating learning-based approaches with formal verification. The light green cluster centers around "formal modeling", "human-robot interaction", and "cyber-physical systems", indicating the relevance of UPPAAL in human-centered applications. Smaller, more specialized clusters, like the blue group focused on "analytical models", "couplings", and "process control", highlight niche application areas. The presence of bridging terms such as "formal methods" and "verification" illustrates the tool's interdisciplinary applicability and its integration into diverse system analysis workflows.

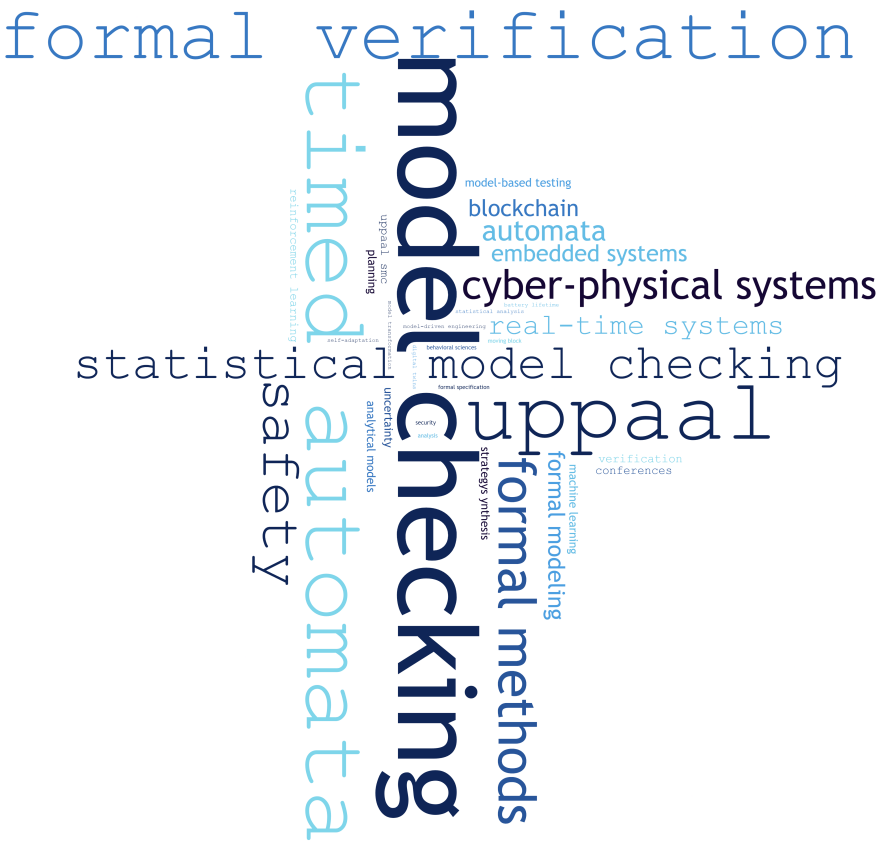

Figure 4. Wordcloud with keywords appearing in the obtained papers.

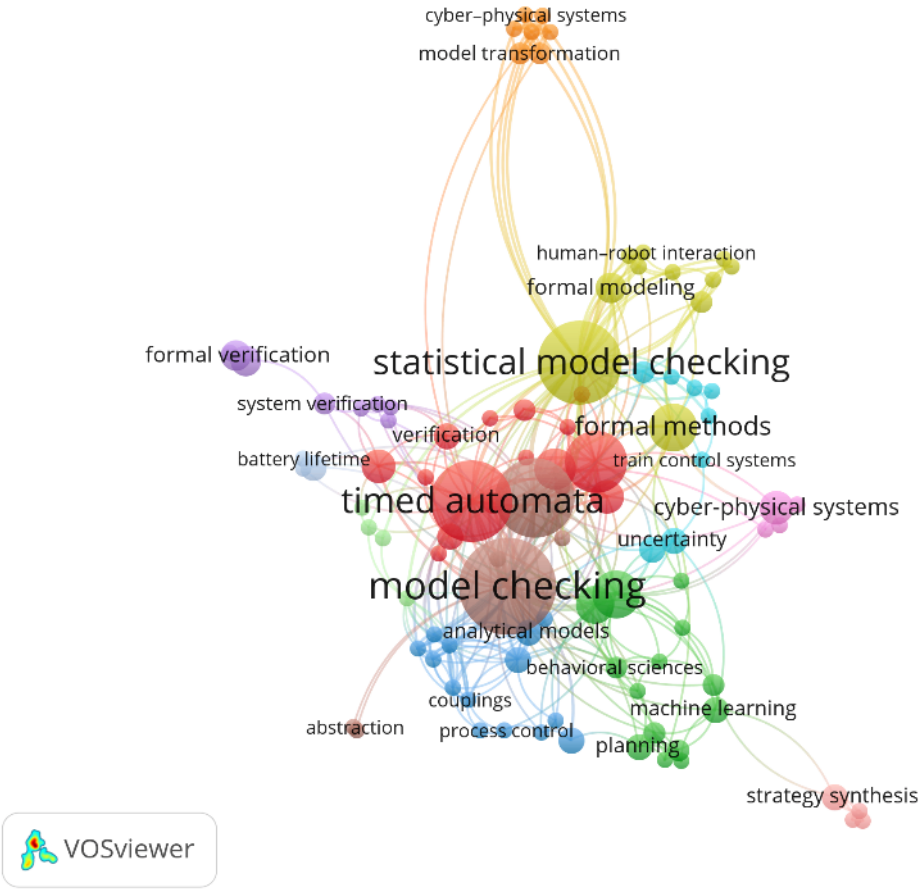

Figure 5. Keyword co-occurrence network visualization.

Another visualization, a keyword density visualization map, is shown in Figure 6. Unlike the co-occurrence network, which emphasizes thematic relationships, the heatmap offers an indication of the prominence of a given keyword within the research landscape. It complements the structural view by offering quantitative visual cues about topic saturation and marginality. The brightest zones reflect strong scholarly focus, revealing where research efforts have been most concentrated. The red region underscores the core methodological focus of the field, while more diffusely colored areas suggest emerging or less-explored niches. Unsurprisingly, the densest areas center around "model checking", "timed automata", and "statistical model checking", confirming their foundational role in the field. Interestingly, the visible separation of dense regions implies that although some topics are conceptually linked, they are investigated with varying degrees of emphasis, pointing to opportunities for interdisciplinary integration or underexploited research directions. Moderate density is observed in the areas linked to "machine learning", "formal methods", and "formal modeling", suggesting growing intersections between classical verification techniques and data-driven or human-centered approaches. In contrast, peripheral topics such as "strategy synthesis" or "human-robot interaction" appear in cooler zones, indicating that while they are connected, they remain niche or less frequently studied within this domain.

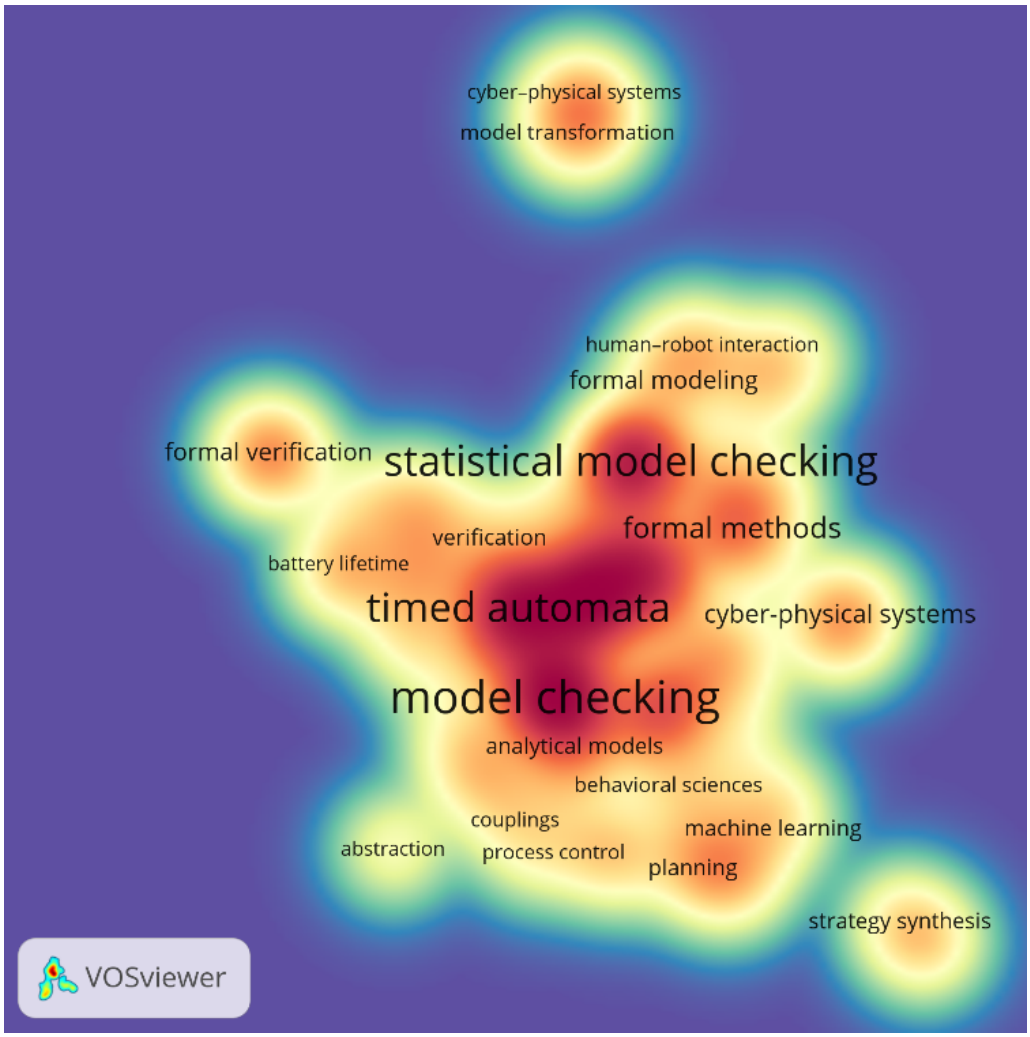

Figure 6. Keyword density visualization map.

4.4. RQ4: What Does the Distribution of Research Papers Regarding Access Options, Scientific Databases, and Types of Publication Look Like?

It was interesting to learn which access option the authors of the papers chose, which scientific databases they chose, and which types of papers they chose to present the results of their research. The distribution of publications in terms of access options is summarized in Figure 7. In line with the global trend of making research results available for a wide range of readers, the proportion of open access articles is significant and currently balances the number of non-open access articles. This parity aligns with the broader trend toward open science, reflecting the growing emphasis on research transparency, accessibility, and public engagement. On the other hand, the continued presence of subscription-based articles indicates that traditional publishing venues still retain influence, particularly in well-established, peer-reviewed journals.

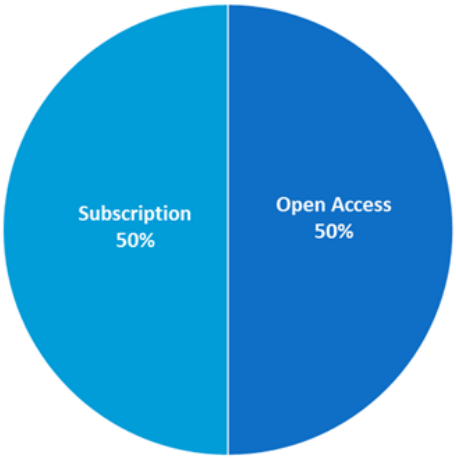

Figure 7. Types of license.

The distribution of publications in terms of the indexing database is summarized in the chart in Figure 8. The three leading scientific databases achieve comparable results and together account for almost 75% of all considered works (IEEE Xplore (25%), Google Scholar (24%), and Springer (23%)). These platforms are widely recognized for their broad visibility. In contrast, the other databases have a much smaller yet notable share (slightly more than 25% of papers in total) (Elsevier (12%), MDPI (8%), and ACM (8%)). The dominant presence of Google Scholar highlights the role of accessible and inclusive indexing in broadening research reach.

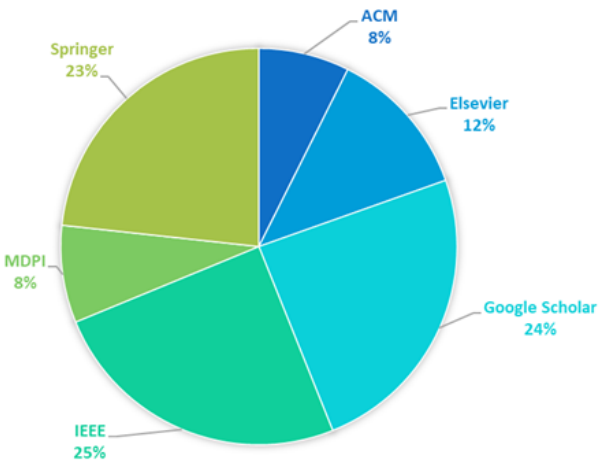

Figure 8. Scientific databases.

Regarding the chosen types of paper, the contributions of conference proceedings and research articles (submitted to journals) were almost equal (49% vs. 48%, respectively), with a small number of book chapters (3%), as shown in Figure 9. This clearly shows that much research is still in progress, as the authors want to present their emerging results during conferences (every second paper using UPPAAL is a conference proceeding). The strong presence of conference papers also reflects the active and fast-evolving nature of formal verification and real-time systems research, where novel approaches and tool extensions are continuously proposed and evaluated. The results of more advanced (or finished), mature, and comprehensive research are usually published as journal articles (likewise, every second paper). Only a small number of works are book chapters.

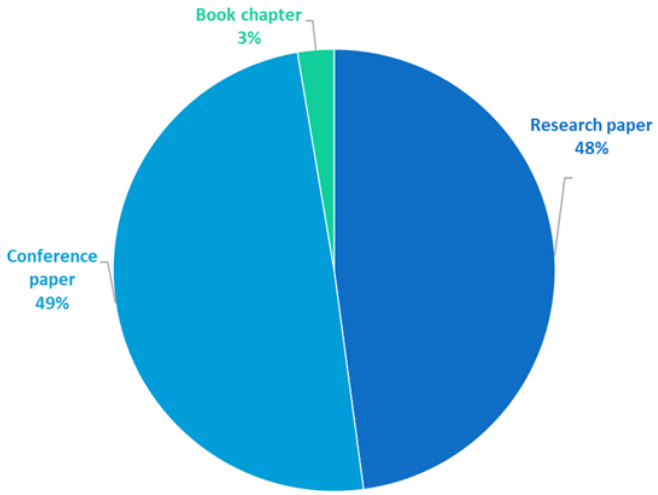

Figure 9. Types of publications.

4.5. RQ5: What Does the Distribution of Research Papers Regarding Geographical Location Look Like?

The distribution of publications in terms of the main research countries is summarized in the map in Figure 10. The geographical distribution of the included works reveals a widespread and globally dispersed interest in the application of the UPPAAL tool. In particular, the regions in which papers are most frequently published are Southeast Asia, Europe, the Americas, and North Africa, with a particularly high density of research output in technologically advanced and research-intensive countries. China emerges as a pioneer in this field (with 32 papers), which reflects both its growing investment in formal methods and strong academic infrastructure. Denmark, with 30 papers, continues to play a pivotal role, likely due to its foundational contribution to the development and maintenance of UPPAAL itself. The other leading countries in this field are Italy (21 papers), France (19 papers), India (15 papers), Sweden (13 papers), Germany (12 papers), the United States of America (11 papers) and Belgium (10 papers). The contributions of the remaining countries are fewer than 10 papers over the considered two-year period. The presence of contributions from regions such as North Africa, South America, and the Middle East highlights the emerging interest and adoption of formal verification tools in developing academic ecosystems. Furthermore, the uneven distribution of publications across countries may also reflect differences in research funding, educational focus, and strategic technological priorities.

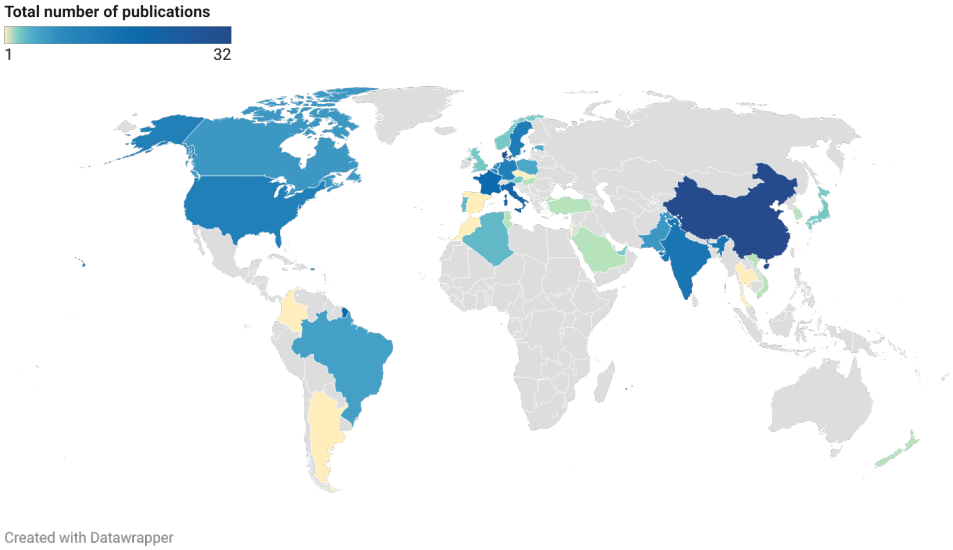

Figure 10. Geographical distribution of papers.

5. Discussion

Most of the papers using UPPAAL take advantage of its capabilities. In justifying its usage, the main aspects mentioned are as follows:

1. Graphical user interface (e.g., [50,58,74,92,97,105,112,131,150,157,205]);
2. Simplicity in model creation (e.g., [50,125]);
3. Powerful simulator and debugger (e.g., [50,97,171]);
4. A powerful verification engine to deliver an absolute guarantee of safety (e.g., [52,106, 125,157,198]);
5. Automatic and thorough verification (e.g., [74]).

These match the initial design criteria for the UPPAAL tool, that is, its efficiency and ease of use. The authors of [105] even noticed that the interface can also help medical professionals who are not familiar with the software to visualize the overall sys, which indicate that it is user-friendly.Timing aspects are important in many approaches, for example, [58], although there also exist ones that do not utilize the notion of time, for example, [50]. An interesting work [169] in the railway domain summarized many advantages of UPPAAL The authors argue that it can be exploited in the requirement compliance phase for the identification and consolidation of both qualitative and quantitative requirements. The authors of [183] describe UPPAAL as the most successful model checker.

5.1. A Brief Comparison with Other Mainstream Formal Validation Tools

The conducted systematic literature review reveals that while UPPAAL excels in real-time and probabilistic verification, formal validation tools like NuSMV [5] and SPIN [7] have strengths in symbolic and software verification, respectively. PRISM [8] provides robust support for probabilistic systems; nuXmv [6] offers an updated platform for symbolic model checking; and tools like HyTech [10], Ymer [11], and Zing [12] are specialized for hybrid and stochastic systems. In addition to the aforementioned general-purpose model checking tools, there are also specialized tools tailored to specific application domains. For instance, AltaRica [211] is a domain-specific modeling language and toolset developed for safety-critical systems, particularly in the aerospace and industrial sectors. While AltaRica is not a general-purpose model checker, its emphasis on safety and reliability analysis—especially in terms of hazard modeling and fault propagation—makes it highly valuable for model-based safety assessment.

The choice of the tool depends on the specific requirements of the system being analyzed, such as the need for real-time constraints, probabilistic behaviors, or software concurrency. UPPAAL is particularly well suited for the modeling and verification of systems in which timing constraints are critical. Its underlying timed automata formalism, coupled with an intuitive graphical interface and efficient verification engine, makes it especially effective for analyzing temporal behaviors under strict time restrictions. Consequently, UPPAAL often demonstrates superior performance and usability compared to general-purpose model checkers when applied to real-time or time-sensitive domains.

5.2. Exploring the Applicability of UPPAAL *Versions*

5.2.1. Classic UPPAAL with Symbolic Model Checking

Capabilities and applicability: The basic UPPAAL tool environment is, first and foremost, a model checking tool. Its main components are the editor, the simulator, and the verifier. The editor allows users to define input models based on timed automata. These automata are enhanced with additional data types, such as integers and arrays. The models are systems of communicating timed automata, which, in th general cases, are non-deterministic. For such a model, a transition system can be generated. The simulator enables the execution of the given model and it can generate execution paths leading to undesired states. The verifier performs symbolic model checking. Unlike the simulator, it explores the entire state spaces and can produce diagnostic traces when necessary. Different kinds of concurrent real-time systems specify the area of typical applicability of this tool.

Demonstrative case study: In [74] a multi-core processor with a shared cache system is analyzed. The challenge is in verifying the cache consistency protocol to ensure data consistency—a non-trivial task in such parallel systems. However, this system (considered at the RTL level) can be conveniently modeled by communicating finite-state machines (automata). To make the model more suitable for analysis, time aspects are included (timed automata are used). This type of model fits well with the capabilities of classic UPPAAL. In the cited work, UPPAAL is used for verification of the protocol by means of model checking. This can be considered a typical case study using basic UPPAAL.

5.2.2. UPPAAL SMC

Capabilities and applicability: UPPAAL SMC, as mentioned before, allows us to perform statistical model checking. It provides a very important practical advantage compared to the classical (symbolic) model checking approach. UPPAAL SMC randomly simulates a series of executions of its models, then performs a statistical analysis of the obtained behaviors. As long as this approach does not generate complete state spaces, it can be applied for systems which have too complex behavior to be handled as a whole. In addition, it is often reasonable (and much easier) to obtain statistical confirmation of the reliability of the system instead of complete formal proof. SMC can obtain, at its input, stochastic time automaton models, which makes it appropriate to model and analyze failures and other extreme or dangerous situations.

Demonstrative case study: In [97], a framework using UPPAAL SMC is presented, in which failure behaviors and attacks are analyzed, allowing for, among others, reliability and cost analyses (including the cost of system repair). A stochastic model called an AFMT (Attack–Fault Maintenance Tree) is developed for this purpose. As a case study, an oil pipeline is considered, where failures such as leakages are analyzed. It can be considered to be a typical application of UPPAAL SMC, together with cyber–physical systems or communication systems, which may be the objects of different kinds of attacks and should also be analyzed to determine the reachability of certain states, etc.

5.2.3. UPPAAL Stratego

Capabilities and applicability: UPPAAL Stratego allows us to develop control strategies with optimized parameters, such as speed, cost, safety, etc., on which a price function depends. In this version of UPPAAL, the models assume the existence of a controller that executes a strategy. Here, as in UPPAAL SMC, stochastic timed models are used, to which statistical model checking can be applied. However, rather than focusing solely on verification, Stratego facilitates the synthesis of optimal strategies through guided simulation. Different learning methods are available, including reinforcement learning approaches like Q-learning and Monte Carlo Tree Search, which are used to synthesize and refine strategies. An obtained strategy (or a set of strategies) may be either deterministic—providing a unique action for each system state—or non-deterministic—offering multiple viable actions from a given state depending on the optimization criteria or trade-offs.

Demonstrative case study: A representative use of UPPAAL Stratego for the control of a heating system based on heat pumps for a family house is presented in [158]. In this scenario, simple simulation and verification of the model were insufficient. Instead, a complex control strategy was required that takes into account the needs of the inhabitants, weather, changing electricity prices, and other parameters. Applying UPPAAL Stratego together with a model called EMDP (based on Markov decision processes) allowed the authors to synthesize an efficient controller, which provides energy savings generally better than those provided by the controllers created by alternative methods. Typical applications of UPPAAL Stratego include the control of complex hybrid systems that exhibit stochastic behavior and operate under dynamic conditions. These applications often require multi-parameter optimization and have been demonstrated in domains such as autonomous robotics, intelligent transportation systems, and adaptive energy management. The synthesis of control strategies represents a significant capability that extends beyond the scope of classical model checking, which is typically limited to verification rather than strategy generation.

5.2.4. UPPAAL TIGA

Capabilities and applicability: The TIGA version shares certain features with UPPAAL Stratego: both use models consisting of a controller and a controlled system, and both aim to synthesize strategies. The system behavior is represented as a transition system with controlled transitions (determined by the controller) and uncontrolled ones. This situation can be seen as a game between two players. Timed automatons are used as the primary modeling elements, and a system is modeled as a composition of such automata. One key difference compared to Stratego is that in TIGA, the automatons are not stochastic, and the resulting strategies are deterministic. UPPAAL TIGA uses a highly efficient symbolic algorithm to solve timed games, typically not requiring full state-space exploration. Solving a game in this context generally means obtaining a winning strategy (or avoiding a losing one), if such a strategy exists. In the games used in TIGA, unlike in the case of UPPAAL Stratego, the aim is not to maximize a continuous cost function, but rather, winning means reaching certain desired states and losing means reaching undesired ones.

Demonstrative case study: In [132,153] a method for synthesizing safe controllers for continuous systems using UPPAAL TIGA is described. Integer-valued bounds for the system variables are derived, and states where the variables are beyond such bounds are considered undesired. UPPAAL TIGA is then used to synthesize a strategy that avoids these states in the corresponding game model. An industrial case study featured in the paper focuses on a stormwater detention pond. This system, like other bounded continuous systems (such as traffic systems), represents a typical application domain for TIGA.

5.2.5. UPPAAL CORA

Capabilities and applicability: UPPAAL CORA (CORA is the abbreviation for cost-optimal reachability analysis) is a tool intended to find, in the state space of a given model, one or more paths to the states that satisfy specified conditions. The paths are optimized according to a cost function. The tool is able to find an optimal path (given sufficient time and memory, since this may require extensive state-space exploration) or to obtain a sub-optimal solution by exploring only part of the state space. UPPAAL CORA builds on the framework of timed automata used in basic UPPAAL, extending it with cost annotations. Is also supports user-defined types and procedures, enhancing modeling flexibility. UPPAAL CORA is particularly well suited for solving scheduling and routing tasks, where cost optimization plays a central role.

Demonstrative case study: The UPPAAL website (https://uppaal.org/, accessed on 17 May 2025) presents several case studies involving UPPAAL CORA, including the vehicle routing problem with time windows (a generalization of the traveling salesman problem with multiple salesmen), the aircraft landing problem, and the energy-optimal task graph scheduling problem. These well-known optimization problems can be effectively addressed by CORA, provided the problem instances are of manageable size. In [190], additional applications are reported, such as job-shop scheduling problems and the power optimization of dataflow applications. Other relevant case studies include programmable logic controllers and smart grid systems. These examples highlight the practical applicability of CORA in domains requiring cost-aware decision-making.

5.2.6. Guidelines for Selecting the Appropriate UPPAAL Version

The diverse capabilities of UPPAAL make it adaptable to a broad range of application domains. However, each version of UPPAAL—be it the classic model checker, the statistical extension (SMC), or the game-based variant (TIGA)—is tailored to address specific modeling needs and verification goals. To support practitioners in selecting the most appropriate version for their context, Table 2 presents a set of general guidelines that summarize the strengths and typical use scenarios of each variant. These guidelines are derived from the analysis of demonstrative case studies and highlight the core features and application conditions that influence tool selection.

Table 2. Guidelines for selecting the appropriate UPPAAL variant.

Use Case/Domain	Recommended Version	Key Features Needed	Notes
Formal verification of real-time systems	Classic	Timed automata, reachability analysis, exhaustive model checking, safety and liveness properties	Ideal for protocol verification, embedded systems, and communication systems
Systems with stochastic behavior or uncertainty	SMC	Statistical model checking, probability evaluation, simulation	Suitable for energy-aware systems, battery analysis, and performance evaluation under uncertainty
Adaptive control in smart systems; resource-aware decision-making; energy-aware scheduling	Stratego	Strategy synthesis, cost optimization, machine learning integration	Suitable for systems requiring optimal and adaptable strategies; leverages reinforcement learning to improve control performance
Adversarial control; planning under uncertainty	TIGA	Timed game automata, strategy synthesis, controller generation	Useful in scheduling, autonomous systems, and human–robot interaction
Real-time scheduling; performance evaluation of timed systems; cost-optimal planning	CORA	Cost variables, optimal scheduling, extended priced timed automata	Ideal for scenarios where timing and resource consumption must be optimized simultaneously

5.3. Open Challenges

Some research projects indicate drawbacks and open challenges; we have done our best to summarize them from the considered literature:

1. The authors of [28] point out a disadvantage of UPPAAL SMC whereby it does not support a hierarchy of states. It is therefore necessary to construct separate templates for the parent-and-child hierarchy in the models used. Despite this fact, the authors still evaluate UPPAAL SMC as a promising tool in estimating the probability of satisfying a user-specified performance query and requires much less checking time than traditional formal verification methods.
2. The authors of [57] report that UPPAAL Stratego solves limited types of objectives, leading it to make too strong assumptions about the problem.
3. The authors of [115] point out that UPPAAL SMC uses the Euler method for solving differential equations, known to be less accurate and entail larger performance overhead in comparison to analytical methods. Moreover, they note that the tool is not optimized for long-lasting simulations.
4. The authors of [131] claim that UPPAAL TIGA (1) cannot process parametric timed automata; (2) has no support for shared memory; and (3) requires each model to be consistent.
5. The authors of [132] point out the following regarding UPPAAL TIGA: (1) it can only calculate the infimum using symbolic methods; (2) its memory usage seems to be the limiting factor in applying the method to large-scale systems.
6. The authors of [149] faced a problem with the machine power needed to validate the given requirements. Indeed, the verification was not completed due to the state-space problem (it crashed after 20 min on one machine, and after 4 h on the other).
7. The authors of [150] indicate the following regardingUPPAAL: (1) it could provide a better user experience (according to chatbot developers); (2) its state machine nature limits the size of flows that can be modeled.
8. The authors of [169] provide a wider discussion of UPPAAL application in the railway domain. They highlight that due to the standardization of the railway process, it is challenging to determine "how to integrate tools and practices [. . .] and how to adapt the overall workflow to accommodate innovation". Moreover, if UPPAAL is meant to be introduced in current industrial processes as T2 tool (the T2 category is dedicated to tools where a fault could lead to an error in verification results), evidence should be provided by the vendors that the results produced by the tool are actually reliable, and that the tool has followed a documented process of development and maintenance. To the knowledge of the authors, this is currently lacking for Uppaal, and this could seriously hamper its adoption.
9. The authors of [170] point out that railway engineers experienced some difficulties in evaluating the results; when UPPAAL provided a counterexample, “it proved almost impossible [. . .] to decipher where the error causing the requirement violation was”. The following solution to this problem is suggested: developing a backward mapping/annotating method to show the counterexample in the high-level model.
10. The authors of [180] noted that system variables cannot change via external interactions with the environment, although some other model checkers enable it, but in these cases, the environment must also be modeled.
11. The authors of [192] mention that the query language for requirement specification in UPPAAL is less expressive than that of Timed Computation Tree Logic (TCTL), and thus, not every TCTL formula can be expressed in UPPAAL. Moreover, they indicate some problems with (1) timed temporal operators; (2) the nesting of model operators; and (3) unavailability of the weak-until operator.

12. The authors of [198] indicate that "the public, academic version [...] is unable to exploit the computing potential of current shared-memory multi-core machines".
13. The authors of [207] state that a limitation in the area of clock synchronization algorithm verification is that UPPAAL does not permit the reading of values of the clock variables.

Despite its strengths, several open challenges are indicated by researchers in the usage of UPPAAL. One of the mentioned limitations is the lack of support for state hierarchies in UPPAAL SMC, requiring separate templates for parent and child states. It also faces some issues with the Euler method for solving differential equations, leading to increased performance overhead and reduced accuracy, particularly in long-duration simulations. UPPAAL Stratego is restricted to a narrow set of objectives, often requiring strong assumptions. UPPAAL SMC and TIGA have limitations such as the inability to process parametric timed automata, a lack of support for shared memory, and scalability challenges related to memory usage. Other concerns include state-space explosion causing verification failures, the inability to interact with system variables externally, and less expressive query languages compared to alternatives like TCTL. Furthermore, UPPAAL struggles to fully utilize modern multi-core processors and lacks the ability to verify clock synchronization algorithms effectively. These challenges highlight the need for further improvements, particularly in scalability, user experience, and domain-specific applications (such as railway systems), which could significantly broaden its utility in real-world, large-scale, and complex systems.

5.4. Possible Solutions

While UPPAAL has proven to be a powerful tool for model checking and the verification of real-time and safety-critical systems, several challenges remain that hinder its broader applicability and efficiency. However, there are several promising solutions that could address these limitations and enhance the tool's performance and usability.

One significant challenge is the lack of support for hierarchical states in UPPAAL SMC. This limitation forces users to create separate templates for parent and child states, complicating model construction. A potential solution would be to extend the modeling framework to incorporate native support for hierarchical state machines, streamlining the modeling process. Similarly to the case with UML state machine diagrams, the use of a hierarchy would offer many new possibilities and more flexibility to the designer. Now, if the specification is written in a hierarchical form, it has to first be "flattened" before it can be processed further [212]. Another open issue is that the use of the Euler method for solving differential equations in UPPAAL SMC is known to introduce significant performance overhead and accuracy issues, particularly in long-duration simulations. One potential improvement could be to integrate more accurate numerical solvers and optimize the tool for parallel and distributed computing. These advancements could improve both the accuracy and efficiency of simulations, especially in complex systems.

Another challenge lies in the limited set of objectives supported by UPPAAL Stratego, which often requires strong assumptions about the system being modeled. To broaden its applicability, future versions of Stratego could integrate more flexible optimization techniques or adopt a general-purpose planning framework, allowing it to handle a wider range of objectives. In the case of UPPAAL TIGA, several limitations affect its ability to handle large-scale systems, such as the inability to process parametric timed automata and a lack of support for shared memory. Enhancing this tool version to address these limitations, together with improvements in memory optimization and symbolic methods, could significantly improve its scalability and versatility in complex systems.

State-space explosion and excessive resource demands have been reported as significant challenges when verifying large systems in UPPAAL. Addressing this issue could involve the incorporation of advanced state-space reduction techniques, such as abstraction or symbolic state-space exploration. Additionally, parallelization strategies and better exploitation of multi-core and distributed computing architectures would help improve performance and reduce computational overhead. In terms of user experience, the modeling approach imposes limitations on the size and complexity of the models that can be handled. Improving the graphical user interface and supporting more flexible modeling paradigms would make the tool more accessible and suitable for large-scale models, thus enhancing the user experience. Another usability issue arises when UPPAAL generates counterexamples that are difficult to interpret, making it challenging for engineers to identify the cause of requirement violations. Developing a backward mapping or annotation feature would allow users to trace counterexamples back to the high-level model, providing greater insight into the verification results.

UPPAAL also currently lacks the ability to interact with system variables externally, limiting its flexibility compared to other model checkers. Extending the tool to allow for external system interaction would make it more suitable for modeling systems that interact with real-world environments, thus increasing its applicability to a wider range of scenarios. Regarding the expressiveness of the query language, it is currently less expressive than Timed Computation Tree Logic (TCTL), which may restrict the types of properties that can be modeled and verified. Enhancing the query language of UPPAAL to support a wider range of temporal operators would provide users with greater flexibility in specifying system requirements. Finally, UPPAAL's inability to fully utilize modern multi-core processors and its reliance on less efficient numerical methods for clock synchronization verification remain significant limitations. Optimizing the underlying algorithms to better exploit multi-core and shared-memory architectures, as well as improving clock synchronization handling, would help address these issues and improve its performance in large, time-sensitive systems.

For domain-specific applications, particularly in the railway sector, UPPAAL has faced integration challenges due to standardization issues and the lack of documented validation processes. Future versions could work on providing a robust framework for integrating UPPAAL with industry standards, with the development of clear documentation regarding the tool's reliability and validation processes. This would help address concerns about its adoption in industrial settings.

These proposed solutions not only address the specific challenges identified in the literature, but also provide a path forward for enhancing the capabilities of the UPPAAL tool in various domains. By implementing these improvements, UPPAAL could further solidify its position as a leading tool for formal verification in real-time and safety-critical systems.

6. Conclusions

This study presents a systematic review of the literature on the application areas of the UPPAAL tool. It has been shown that its comprehensive features make it suitable for potential use in various fields. The study included 188 articles published in 2022 and 2023. The results clearly indicate that the most popular version is UPPAAL SMC, which supports statistical model checking, followed by UPPAAL Stratego, dedicated to strategy analysis. The distribution of works between conference papers and research papers is almost equal, which suggests that many research projects are still ongoing (the preliminary results are usually first presented at conferences). This is very promising for the near future. The most frequently publishing regions are Southeast Asia, Europe, and the Americas.

Five research questions were defined in this study, corresponding to (1) the application areas of the UPPAAL tool, (2) the popularity of its versions, (3) the most popular keywords in obtained papers, (4) the distribution of articles regarding access options, scientific databases, and publication types, and (5) the distribution of papers in terms of geographical location. All of these questions have been thoroughly answered in this paper. This allows us to provide summaries and insights into possible further developments of the UPPAAL tool. The literature analysis shows that the choice of the UPPAAL tool often results from its ease of use and high efficiency. These aspects are often emphasized in various application areas, since UPPAAL is frequently used by non-engineers. This aligns with the original design goals of UPPAAL, prioritizing user accessibility and computational performance. For instance, studies have highlighted the intuitive graphical interface as beneficial not only to software engineers but also to medical professionals, enabling broader interdisciplinary collaboration. It follows that these aspects should still be the key design issues for developers. Some of the research papers point out the imperfections of the tool, such as limited support for hierarchical modeling, restricted objective types in strategy synthesis, higher performance overhead due to numerical methods, and challenges with scalability and usability. This information may be of great importance to programmers and engineers involved in software development.

The main limitation of this study is that only publications written in English were taken into account. Some preliminary results published in other languages, for example, those presented at local conferences, may therefore have been omitted.

Author Contributions: Conceptualization, I.G.; methodology, I.G.; investigation, K.G. and I.G.; writing—original draft preparation, K.G. and I.G.; writing—review and editing, I.G. and A.K.; visualization, K.G.; supervision, I.G.; project administration, I.G.; funding acquisition, I.G. All authors have read and agreed to the published version of the manuscript.

Funding: This work was partially supported by the program of the Polish Ministry of Science under the title 'Regional Excellence Initiative', project no. RID/SP/0050/2024/1.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Woodcock, J.; Larsen, P.G.; Bicarregui, J.; Fitzgerald, J. Formal methods: Practice and experience. *ACM Comput. Surv.* **2009**, *41*. [CrossRef]
2. Edmund, M.; Clarke, J.; Grumberg, O.; Peled, D.A. *Model Checking*; MIT Press: Cambridge, MA, USA, 1999; p. 314.
3. Karmakar, R. Symbolic Model Checking: A Comprehensive Review for Critical System Design. In *Proceedings of the Advances in Data and Information Sciences*; Tiwari, S., Trivedi, M.C., Kolhe, M.L., Mishra, K., Singh, B.K., Eds.; Springer: Singapore, 2022; pp. 693–703.
4. Legay, A.; Viswanathan, M. Statistical model checking: Challenges and perspectives. *Int. J. Softw. Tools Technol. Transf.* **2015**, *17*, 369–376. [CrossRef]
5. Cimatti, A.; Clarke, E.; Giunchiglia, F.; Roveri, M. NuSMV: A new symbolic model checker. *Int. J. Softw. Tools Technol. Transf.* **2000**, *2*, 410–425. [CrossRef]
6. Cavada, R.; Cimatti, A.; Dorigatti, M.; Griggio, A.; Mariotti, A.; Micheli, A.; Mover, S.; Roveri, M.; Tonetta, S. The nuXmv symbolic model checker. In Proceedings of the Computer Aided Verification: 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, 18–22 July 2014; Proceedings 26; Springer: Berlin/Heidelberg, Germany, 2014; pp. 334–342.
7. Holzmann, G.J. The model checker SPIN. *IEEE Trans. Softw. Eng.* **1997**, *23*, 279–295. [CrossRef]
8. Kwiatkowska, M.; Norman, G.; Parker, D. PRISM: Probabilistic symbolic model checker. In Proceedings of the International Conference on Modelling Techniques and Tools for Computer Performance Evaluation, London, UK, 14–17 April 2002; Springer: Berlin/Heidelberg, Germany, 2002; pp. 200–204.

9. Behrmann, G.; David, A.; Larsen, K.G. A tutorial on UPPAAL. In *Formal Methods for the Design of Real-Time Systems*; Springer: Berlin/Heidelberg, Germany, 2004; pp. 200–236.
10. Henzinger, T.A.; Ho, P.H.; Wong-Toi, H. HyTech: A model checker for hybrid systems. In Proceedings of the Computer Aided Verification: 9th International Conference, CAV'97, Haifa, Israel, 22–25 June 1997; Proceedings 9; Springer: Berlin/Heidelberg, Germany, 1997; pp. 460–463.
11. Younes, H.L. Ymer: A statistical model checker. In Proceedings of the International Conference on Computer Aided Verification, Edinburgh, UK, 6–10 July 2005; Springer: Berlin/Heidelberg, Germany, 2005; pp. 429–433.
12. Andrews, T.; Qadeer, S.; Rajamani, S.K.; Rehof, J.; Xie, Y. Zing: A model checker for concurrent software. In Proceedings of the Computer Aided Verification: 16th International Conference, CAV 2004, Boston, MA, USA, 13–17 July 2004; Proceedings 16; Springer: Berlin/Heidelberg, Germany, 2004; pp. 484–487.
13. Shkarupylo, V.V.; Blinov, I.V.; Chemeris, A.A.; Dusheba, V.V.; Alsayaydeh, J.A.J. On Applicability of Model Checking Technique in Power Systems and Electric Power Industry. In *Systems, Decision and Control in Energy III*; Zaporozhets, A., Ed.; Springer International Publishing: Cham, Switzerland, 2022; pp. 3–21. [CrossRef]
14. Castiglioni, V.; Lanotte, R.; Loreti, M.; Tini, S. Evaluating the Effectiveness of Digital Twins Through Statistical Model Checking with Feedback and Perturbations. In *Proceedings of the Formal Methods for Industrial Critical Systems*; Haxthausen, A.E., Serwe, W., Eds.; Springer: Cham, Switzerland, 2024; pp. 21–39.
15. Alwhishi, G.; Bentahar, J.; Elwhishi, A.; Pedrycz, W.; Drawel, N. Multi-valued model checking IoT and intelligent systems with commitment protocols in multi-source data environments. *Inf. Fusion* **2024**, *102*, 102048. [CrossRef]
16. Khan, N.; Nauman, M.; Almadhor, A.S.; Akhtar, N.; Alghuried, A.; Alhudhaif, A. Guaranteeing Correctness in Black-Box Machine Learning: A Fusion of Explainable AI and Formal Methods for Healthcare Decision-Making. *IEEE Access* **2024**, *12*, 90299–90316. [CrossRef]
17. Zhou, W.; Zhao, Y.; Zhang, Y.; Wang, Y.; Yin, M. A comprehensive survey of UPPAAL-assisted formal modeling and verification. *Softw. Pract. Exp.* **2025**, *55*, 272–297. [CrossRef]
18. David, A.; Larsen, K.G.; Legay, A.; Mikučionis, M.; Poulsen, D.B. Uppaal SMC tutorial. *Int. J. Softw. Tools Technol. Transf.* **2015**, *17*, 397–415. [CrossRef]
19. David, A.; Jensen, P.G.; Larsen, K.G.; Mikučionis, M.; Taankvist, J.H. Uppaal stratego. In Proceedings of the Tools and Algorithms for the Construction and Analysis of Systems: 21st International Conference, TACAS 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, 11–18 April 2015; Proceedings 21; Springer: Berlin/Heidelberg, Germany, 2015; pp. 206–211.
20. Behrmann, G.; Cougnard, A.; David, A.; Fleury, E.; Larsen, K.G.; Lime, D. *UPPAAL TIGA User-Manual;* Aalborg University: Aalborg, Denmark, 2007.
21. Sarkis-Onofre, R.; Catalá-López, F.; Aromataris, E.; Lockwood, C. How to properly use the PRISMA Statement. *Syst. Rev.* **2021**, *10*, 1–3. [CrossRef]
22. Guldstrand Larsen, K.; Lorber, F.; Nielsen, B. 20 Years of Real Real Time Model Validation. In *Proceedings of the Formal Methods*; Havelund, K., Peleska, J., Roscoe, B., de Vink, E., Eds.; Springer: Cham, Switzerland, 2018; pp. 22–36.
23. Gu, R.; Enoiu, E. Model-Based Policy Synthesis and Test-Case Generation for Autonomous Systems. In Proceedings of the IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), Dublin, Ireland, 16–20 April 2023; pp. 18–27. [CrossRef]
24. Huang, Z.; Li, B.; Du, D.; Li, Q. A Model Checking Based Approach to Detect Safety-Critical Adversarial Examples on Autonomous Driving Systems. In *Proceedings of the International Colloquium on Theoretical Aspects of Computing*; Springer: Berlin/Heidelberg, Germany, 2022; pp. 238–254.
25. Hou, Z.; Wang, S.; Liu, H.; Yang, Y.; Zhang, Y. Twin Scenarios Establishment for Autonomous Vehicle Digital Twin Empowered SOTIF Assessment. *IEEE Trans. Intell. Veh.* **2023**, *9*, 1965–1976. [CrossRef]
26. Wang, M.; Li, T.; Liu, J.; Dou, H.; Chen, H.; Zhang, J.; Zhang, L. Modeling and Verification of Autonomous Driving Systems under Stochastic Spatio-Temporal Constraints. In Proceedings of the International Conference on Software Engineering and Knowledge Engineering, Virtual, 1–10 July 2023. Available online: https://ksiresearch.org/seke/seke23paper/paper221.pdf (accessed on 28 May 2025).
27. Wang, J.; Huang, Z.; Zhu, Y.; Shen, G. Statistical Model Checking for Stochastic and Hybrid Autonomous Driving Based on Spatio-Clock Constraints. *Int. J. Softw. Eng. Knowl. Eng.* **2022**, *32*, 553–582. [CrossRef]
28. Liu, H.; Liu, J.; Sun, H.; Li, T.; Zhang, J. Uncertainty-Aware Behavior Modeling and Quantitative Safety Evaluation for Automatic Flight Control Systems. In Proceedings of the IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS), Guangzhou, China, 5–9 December 2022; pp. 549–560. [CrossRef]
29. Ganguly, R.; Xue, Y.; Jonckheere, A.; Ljung, P.; Schornstein, B.; Bonakdarpour, B.; Herlihy, M. Distributed runtime verification of metric temporal properties for cross-chain protocols. In Proceedings of the IEEE 42nd International Conference on Distributed Computing Systems (ICDCS), Bologna, Italy, 10–13 July 2022; IEEE: New York, NY, USA, 2022; pp. 23–33.

30. Park, W.S.; Lee, H.; Choi, J.Y. Formal Modeling of Smart Contract-based Trading System. In Proceedings of the 24th International Conference on Advanced Communication Technology (ICACT), Pyeongchang, Republic of Korea, 13–16 February 2022; pp. 48–52. [CrossRef]
31. Liu, Y.; Li, X.; Ma, Y. FGAC: A Fine-Grained Access Control Framework for Supply Chain Data Sharing. *Systems* **2022**, *10*, 208. [CrossRef]
32. Hammami, M.A.; Lahami, M.; Maâlej, A.J. Towards a Dynamic Testing Approach for Checking the Correctness of Ethereum Smart Contracts. In Proceedings of the International Conference on Risks and Security of Internet and Systems, Sousse, Tunisia, 7–9 December 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 85–100.
33. Barati, M.; Adu-Duodu, K.; Rana, O.; Aujla, G.S.; Ranjan, R. Compliance checking of cloud providers: Design and implementation. *Distrib. Ledger Technol. Res. Pract.* **2023**, *2*, 1–20. [CrossRef]
34. Kovalov, A.; Patil, G.; Bansal, V.; Gerndt, A. Model checking message delivery times in SpaceWire networks. In Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings, Montreal, QC, Canada, 23–28 October 2022; pp. 267–275.
35. Chen, N.; Zhu, H. IoT Modeling and Verification: From the CaIT Calculus to UPPAAL. *IEICE Transations Inf. Syst.* **2023**, *106*, 1507–1518. [CrossRef]
36. V, G.L.; Pillai, A.S.; Raj, A. Modeling & Verification of an Adaptive IoT System using Uppaal. In Proceedings of the IEEE 3rd Global Conference for Advancement in Technology (GCAT), Bangalore, India, 7–9 October 2022; pp. 1–5. [CrossRef]
37. Naeem, M.; Albano, M.; Magrin, D.; Nielsen, B.; Guldstrand, K. A Sigfox Module for the Network Simulator 3. In Proceedings of the Workshop on Ns-3, New York, NY, USA, 22–23 June 2022; WNS3 '22; pp. 81–88. [CrossRef]
38. Naeem, M.; Albano, M.; Larsen, K.G.; Nielsen, B.; Høedholt, A.; Laursen, C. Modelling and Analysis of a Sigfox-Based IoT Network Using uppaalsmc. *IEEE Sens. J.* **2023**, *23*, 10577–10587. [CrossRef]
39. Jawad, A.; Newton, L.; Matrawy, A.; Jaskolka, J. A Formal Analysis of the Efficacy of Rebooting as a Countermeasure Against IoT Botnets. In Proceedings of the ICC-IEEE International Conference on Communications, Seoul, Republic of Korea, 16–20 May 2022; pp. 2206–2211. [CrossRef]
40. Bezerra, W.R.; Martina, J.E.; Westphall, C.B. A Formal Verification of a Reputation Multi-Factor Authentication Mechanism for Constrained Devices and Low-Power Wide-Area Network Using Temporal Logic. *Sensors* **2023**, *23*, 6933. [CrossRef]
41. Backeman, P.; Kunnappilly, A.; Seceleanu, C. Supporting 5G service orchestration with formal verification. *Comput. Sci. Inf. Syst.* **2023**, *20*, 329–357. [CrossRef]
42. Roumane, A.; Kechar, B. A statistical model checking approach to analyse the random access protocol. *Int. J. Wirel. Mob. Comput.* **2022**, *23*, 338–349. [CrossRef]
43. Asokan, S.; Kumar, G.S. Formal modeling of the gPTP clock synchronization algorithm in automotive ethernet. *Innov. Syst. Softw. Eng.* **2023**, *19*, 265–281. [CrossRef]
44. de Moraes, R.M.; Sup, L.M.A.; Bauchspiess, A. TCP-Puerto-Londero: A New Approach for End-to-End Queue Length Control. *J. Commun. Inf. Syst.* **2023**, *38*, 105–114.
45. Alshalalfah, A.L.; Mohamed, O.A.; Ouchani, S. A framework for modeling and analyzing cyber-physical systems using statistical model checking. *Internet Things* **2023**, *22*, 100732. [CrossRef]
46. Kröger, J.; Fränzle, M. Updates at Runtime for Cyber Physical Systems. A Game Theoretic Approach. In Proceedings of the Software Engineering Workshops, Stuttgart, Germany, 19 February 2019; Gesellschaft für Informatik eV: Berlin, Germany, 2023.
47. Anto, K.; Swain, A.; Roop, P. A novel framework for the design of resilient cyber-physical systems using control theory and formal methods. *IEEE Access* **2023**, *11*, 73556–73567. [CrossRef]
48. Ghosh, P.; Karsai, G. Distributed Cyber Physical Systems Software Model Checking using Timed Automata. In Proceedings of the IEEE 26th International Symposium on Real-Time Distributed Computing (ISORC), Nashville, TN, USA, 23–25 May 2023; pp. 164–169. [CrossRef]
49. Santos, A.A.; da Silva, A.F.; Pereira, F. Simulation of Cyber-Physical Intelligent Mechatronic Component Behavior Using Timed Automata Approach. In Proceedings of the International Conference Innovation in Engineering, Minho, Portugal, 28–30 June 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 72–85.
50. Hansen, S.T.; Thule, C.; Gomes, C.; van de Pol, J.; Palmieri, M.; Inci, E.O.; Madsen, F.; Alfonso, J.; Castellanos, J.A.; Rodriguez, J.M. Verification and synthesis of co-simulation algorithms subject to algebraic loops and adaptive steps. *Int. J. Softw. Tools Technol. Transf.* **2022**, *24*, 999–1024. [CrossRef]
51. Miedema, L.; Grelck, C. Strategy Switching: Smart Fault-Tolerance for Weakly-Hard Resource-Constrained Real-Time Applications. In Proceedings of the International Conference on Software Engineering and Formal Methods, Berlin, Germany, 26–30 September 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 129–145.
52. Boudjadar, J.; Tomko, M. A Digital Twin Setup for Safety-aware Optimization of a Cyber-physical System. In Proceedings of the 19th International Conference on Informatics in Control, Automation and Robotics, Lisbon, Portugal, 14–16 July 2022; pp. 161–168. Available online: https://www.scitepress.org/Papers/2022/112721/112721.pdf (accessed on 28 May 2025).

53. Delimpaltadakis, G.; de Albuquerque Gleizer, G.; Van Straalen, I.; Mazo, M., Jr. ETCetera: Beyond event-triggered control. In Proceedings of the 25th ACM International Conference on Hybrid Systems: Computation and Control, Milan, Italy, 4–6 May 2022; pp. 1–11.
54. Lois, R.S.; Cole, D.G. Designing Secure and Resilient Cyber-Physical Systems Using Formal Models. In Proceedings of the 2022 Resilience Week (RWS), National Harbor, MD, USA, 26–29 September 2022; IEEE: New York, NY, USA, 2022; pp. 1–6.
55. Ali, A.T.; Gruska, D. Dynamic attack trees methodology. In *Proceedings of the Interdisciplinary Research in Technology and Management (IRTM)*; IEEE: New York, NY, USA, 2022; pp. 1–9.
56. Ashraf, K.; Le Moullec, Y.; Pardy, T.; Rang, T. Design of Cyber Bio-analytical Physical Systems: Formal methods, architectures, and multi-system interaction strategies. *Microprocess. Microsyst.* **2023**, *97*, 104780. [CrossRef]
57. Ballot, G.; Malvone, V.; Leneutre, J.; Borde, E. Reasoning about Moving Target Defense in Attack Modeling Formalisms. In Proceedings of the 9th ACM Workshop on Moving Target Defense, Los Angeles, CA, USA, 7 November 2022; pp. 55–65.
58. AlQadheeb, A.; Bhattacharyya, S.; Perl, S. Enhancing cybersecurity by generating user-specific security policy through the formal modeling of user behavior. *Array* **2022**, *14*, 100146. [CrossRef]
59. Sakata, K.; Fujita, S.; Sawada, K.; Iwasawa, H.; Endoh, H.; Matsumoto, N. Model verification of fallback control system under cyberattacks via UPPAAL. *Adv. Robot.* **2023**, *37*, 156–168. [CrossRef]
60. Murthy, K.R.; Kumar, S.; Kumar Singh, M. Cyber Physical Systems (CPS) Security Verification Using Model Checking. In *Recent Developments in Electronics and Communication Systems: Proceedings of the First International Conference on Recent Developments in Electronics and Communication Systems (RDECS-2022)*; IOS Press: Amsterdam, The Netherlands, 2023; Volume 32, p. 234.
61. Hafeez, S.; Atif, M.; Naseer, M. Formal Specification and Verification of Distributed Denial of Service (DDoS). *VAWKUM Trans. Comput. Sci.* **2022**, *10*, 132–142. [CrossRef]
62. Sakata, K.; Fujita, S.; Sawada, K. Model Verification of Resilient Third-Party Monitoring System Against Cyberattacks. In Proceedings of the 2022 IEEE International Conference on Consumer Electronics (ICCE), Las Vegas, NV, USA, 7–9 January 2022; pp. 1–6. [CrossRef]
63. Wang, Y.; Zhou, Q.; Zhang, Y.; Zhang, X.; Du, J. A formal modeling and verification scheme with an RNN-based attacker for CAN communication system Authenticity. *Electronics* **2022**, *11*, 1773. [CrossRef]
64. Li, D.; Zhang, Q.; Zhao, D.; Li, L.; He, J.; Yuan, Y.; Zhao, Y. Hardware Trojan Detection Using Effective Property-Checking Method. *Electronics* **2022**, *11*, 2649. [CrossRef]
65. Kumar, R.; Singh, S.; Narra, B.; Kela, R. Co-engineering Safety-Security Using Statistical Model Checking. In Proceedings of the International Conference on Formal Techniques for Distributed Objects, Components, and Systems, Lucca, Italy, 13–16 June 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 88–92.
66. Lanotte, R.; Merro, M.; Zannone, N. Impact Analysis of Coordinated Cyber-Physical Attacks via Statistical Model Checking: A Case Study. In Proceedings of the International Conference on Formal Techniques for Distributed Objects, Components, and Systems, Lisbon, Portugal, 19–23 June 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 75–94.
67. Kiviriga, A.; Larsen, K.G.; Nyman, U. Randomized reachability analysis in UPPAAL: Fast error detection in timed systems. *Int. J. Softw. Tools Technol. Transf.* **2022**, *24*, 1025–1042. [CrossRef]
68. Cuartas, J.; Aranda, J.; Cordy, M.; Ortiz, J.; Perrouin, G.; Schobbens, P.Y. MUPPAAL: Reducing and Removing Equivalent and Duplicate Mutants in UPPAAL. In Proceedings of the IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), Dublin, Ireland, 16–20 April 2023; pp. 52–61. [CrossRef]
69. Arora, S.; Hansen, R.R.; Larsen, K.G.; Legay, A.; Poulsen, D.B. Statistical model checking for probabilistic hyperproperties of real-valued signals. In *Proceedings of the International Symposium on Model Checking Software*; Springer: Berlin/Heidelberg, Germany, 2022; pp. 61–78.
70. Oh, B.; Ahn, J.; Bae, S.; Son, M.; Lee, Y.; Kang, M.; Kim, Y. Preventing SIM Box Fraud Using Device Model Fingerprinting. In Proceedings of the NDSS Symposium, San Diego, CA, USA, 27 February–3 March 2023.
71. Neupane, R.; Mehrpouyan, H. An ontology-based framework for formal verification of safety and security properties of control logics. In Proceedings of the 14th International Conference on Electronics, Computers and Artificial Intelligence (ECAI), Ploiesti, Romania, 30 June–1 July 2022; IEEE: New York, NY, USA, 2022; pp. 1–8.
72. Campusano, M.; Hacks, S.; Kang, E. Towards model driven safety and security by design. In Proceedings of the International Workshops 10th QuASoQ 2022 & the 6th (SEED 2022) Co-Located with 29th APSEC22, Virtual, 6 December 2022; Series CEUR Workshop Proceedings; Volume 3330, pp. 34–41.
73. Jamroga, W.; Kurpiewski, D.; Malvone, V. How to measure usable security: Natural strategies in voting protocols. *J. Comput. Secur.* **2022**, *30*, 381–409.

74. Zhao, Y.; Shi, B.; Zhang, Q.; Yuan, Y.; He, J. Research on Cache Coherence Protocol Verification Method Based on Model Checking. *Electronics* **2023**, *12*, 3420. [CrossRef]
75. Christensen, M.; Tzimpragos, G.; Kringen, H.; Volk, J.; Sherwood, T.; Hardekopf, B. PyLSE: A Pulse-Transfer Level Language for Superconductor Electronics. In Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, 13–17 June 2022; PLDI 2022; pp. 671–686. [CrossRef]
76. Beck, T.; Boniol, F.; Ermont, J.; Wartel, F.; Maillet, L. An automata-based method for interference analysis in multi-core processors. In Proceedings of the 15th Junior Researcher Workshop on Real-Time Computing (JRWRTC 2022)@ RTNS 2022, Paris, France, 7–8 June 2022; pp. 1–4. Available online: https://hal.science/hal-03857409/document (accessed on 28 May 2025).
77. Ke, Y.; Xia, X. Timed Automaton-Based Quantitative Feasibility Analysis of Symmetric Cipher in Embedded RTOS: A Case Study of AES. *Secur. Commun. Netw.* **2022**, *2022*, 4118994.
78. Scheipel, T.; Batista Ribeiro, L.; Sagaster, T.; Baunach, M. Smartos: An OS architecture for sustainable embedded systems. In *Proceedings of the Tagungsband des FG-BS Frühjahrstreffens 2022*; Gesellschaft für Informatik eV: Berlin, Germany, 2022; pp. 10–18420.
79. Ribeiro, L.B.; Lorber, F.; Nyman, U.; Larsen, K.G.; Baunach, M. A Modeling Concept for Formal Verification of OS-Based Compositional Software. In Proceedings of the International Conference on Fundamental Approaches to Software Engineering, Paris, France, 22–27 April 2023; Springer: Cham, Switzerland, 2023; pp. 26–46.
80. Yang, X.; Chen, X.; Wang, J. A Model Checking Based Software Requirements Specification Approach for Embedded Systems. In Proceedings of the IEEE 31st International Requirements Engineering Conference Workshops (REW), Hannover, Germany, 4–5 September 2023; IEEE: New York, NY, USA, 2023; pp. 184–191.
81. Ding, G.; Liu, J. SysML Flow Model. In Proceedings of the 29th Asia-Pacific Software Engineering Conference (APSEC), Virtual, 6–9 December 2022; IEEE: New York, NY, USA, 2022; pp. 159–168.
82. Jesus, V.S.d.; Lazarin, N.M.; Pantoja, C.E.; Manoel, F.C.P.B.; Alves, G.V.; Viterbo, J. A middleware for providing communicability to Embedded MAS based on the lack of connectivity. *Artif. Intell. Rev.* **2023**, *3*, 2971–3001. [CrossRef]
83. Gu, R.; Jensen, P.G.; Seceleanu, C.; Enoiu, E.; Lundqvist, K. Correctness-guaranteed strategy synthesis and compression for multi-agent autonomous systems. *Sci. Comput. Program.* **2022**, *224*, 102894.
84. Gu, R.; Jensen, P.G.; Poulsen, D.B.; Seceleanu, C.; Enoiu, E.; Lundqvist, K. Verifiable strategy synthesis for multiple autonomous agents: A scalable approach. *Int. J. Softw. Tools Technol. Transf.* **2022**, *24*, 395–414. [CrossRef]
85. Jamroga, W.; Kim, Y. Practical Abstraction for Model Checking of Multi-Agent Systems. In Proceedings of the 20th International Conference on Principles of Knowledge Representation and Reasoning Main Track, Rhodes, Greece, 2–8 September 2023. [CrossRef]
86. Yousaf, S.; Haque, H.M.U.; Atif, M.; Hashmi, M.A.; Khalid, A.; Vinh, P.C. A context-aware multi-agent reasoning based intelligent assistive formalism. *Internet Things* **2023**, *23*, 100857.
87. Ribeiro, L.B.; Nagarajan, D.; Manjunath, V.; Ali Ahmad, M.T.; Baunach, M. Verifying Liveness and Real-Time of OS-Based Embedded Software. In Proceedings of the 25th Euromicro Conference on Digital System Design (DSD), Maspalomas, Spain, 31 August–2 September 2022; pp. 679–688. [CrossRef]
88. Wang, J.; Wu, X.; Hou, G.; Li, P.; Gao, A.; Chen, Z.; Gao, H. Modeling and reliability verification of industrial control network protocol based on time state transition matrix. *Int. J. Commun. Syst.* **2022**, *35*, e5140. [CrossRef]
89. Zhu, X. Collaborative modelling and simulation for manufacturing cost analysis. *Int. J. Simul. Model. (IJSIMM)* **2023**, *22*, 338–349. [CrossRef]
90. Liu, S.; Gao, Z. Modeling and Verification of Intelligent Manufacturing Product Line System with Timed Automata. In Proceedings of the International Conference on Intelligent Manufacturing, Advanced Sensing and Big Data (IMASBD), Guilin, China, 22–24 July 2022; pp. 1–6. . [CrossRef]
91. Zhang, C.; Zhou, G.; Jing, Y.; Wang, R.; Chang, F. A digital twin-based automatic programming method for adaptive control of manufacturing cells. *IEEE Access* **2022**, *10*, 80784–80793.
92. Ozkan, M.; Demirci, Z.; Aslan, Ö.; Yazıcı, A. Safety Verification of Multiple Industrial Robot Manipulators with Path Conflicts Using Model Checking. *Machines* **2023**, *11*, 282. [CrossRef]
93. Himmiche, S.; Marangé, P.; Aubry, A.; Pétin, J.F. Robustness Evaluation Process for Scheduling under Uncertainties. *Processes* **2023**, *11*, 371. [CrossRef]
94. Tahiri, I.; Philippot, A.; Carré-Ménétrier, V.; Tajer, A. A fault-tolerant and a reconfigurable control framework: Application to a real manufacturing system. *Processes* **2022**, *10*, 1266. [CrossRef]
95. Siboulet, É.; Pottier, L.; Ranger, T.; Riera, B. Fresh Approaches for Structured Text Programmable Logic Controllers Programs Verification. *Processes* **2023**, *11*, 687. [CrossRef]
96. Ukegbu, C.; Neupane, R.; Mehrpouyan, H. Ontology-Based Framework for Boundary Verification of Safety and Security Properties in Industrial Control Systems. In Proceedings of the 2023 European Interdisciplinary Cybersecurity Conference, Stavanger, Norway, 14–15 June 2023; EICC '23; pp. 47–52. [CrossRef]

97. Kumar, R.; Narra, B.; Kela, R.; Singh, S. AFMT: Maintaining the safety-security of industrial control systems. *Comput. Ind.* **2022**, *136*, 103584. [CrossRef]
98. Larsson, J.; Enoiu, E.P. Test Generation and Mutation Analysis of Energy Consumption using UPPAAL SMC and MATS. In Proceedings of the IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), Dublin, Ireland, 16–20 April 2023; IEEE: New York, NY, USA, 2023; pp. 186–189.
99. Jawad, A.; Jaskolka, J. Defense Models for Data Recovery in Industrial Control Systems. In Proceedings of the International Symposium on Foundations and Practice of Security, Ottawa, ON, Canada, 12–14 December 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 271–286. Available online: https://link.springer.com/conference/fps (accessed on 28 May 2025).
100. Kong, L.; Yang, Q.; Zhou, Q.; Xing, J.; Sun, X.; Zou, R. Embedding knowledge into BIM: A case study of extending BIM with firefighting plans. *J. Build. Eng.* **2022**, *49*, 103999. [CrossRef]
101. Bakhshi, Z.; Rodriguez-Navas, G.; Hansson, H. Analyzing the performance of persistent storage for fault-tolerant stateful fog applications. *J. Syst. Archit.* **2023**, *144*, 103004. [CrossRef]
102. Jensen, P.G.; Larsen, K.G.; Mikučionis, M. Playing Wordle with Uppaal Stratego. In *A Journey from Process Algebra via Timed Automata to Model Learning: Essays Dedicated to Frits Vaandrager on the Occasion of His 60th Birthday*; Springer: Berlin/Heidelberg, Germany, 2022; pp. 283–305.
103. Dierl, S.; Howar, F.M.; Kauffman, S.; Kristjansen, M.; Guldstrand Larsen, K.; Lorber, F.; Mauritz, M. Learning Symbolic Timed Models from Concrete Timed Data. In Proceedings of the NASA Formal Methods Symposium, Houston, TX, USA, 16–18 May 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 104–121.
104. Quin, F.; Weyns, D.; Gheibi, O. Reducing large adaptation spaces in self-adaptive systems using classical machine learning. *J. Syst. Softw.* **2022**, *190*, 111341.
105. Parveen, R.; Goveas, N. Transforming Medical Resource Utilization Process to Verifiable Timed Automata Models in Cyber-Physical Systems. In Proceedings of the International Conference on Distributed Computing and Internet Technology, Bhubaneswar, India, 19–23 January 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 111–126.
106. Baranov, E.; Bowles, J.; Given-Wilson, T.; Legay, A.; Webber, T. A Secure User-Centred Healthcare System: Design and Verification. In Proceedings of the 10th International Symposium: From Data to Models and Back, Virtual Event, 6–7 December 2021; pp. 44–60.
107. Fayad, M.; Mostefaoui, A.; Chouali, S.; Benbernou, S. Toward a design model-oriented methodology to ensure QoS of a cyber-physical healthcare system. *Computing* **2022**, *104*, 1615–1641. [CrossRef]
108. Baird, A.; Pinisetty, S.; Allen, N.; Patel, N.; Roop, P. Runtime Verification for Clinically Interpretable Arrhythmia Classification. In Proceedings of the 20th ACM-IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE), Shanghai, China, 13–14 October 2022; IEEE: New York, NY, USA, 2022; pp. 1–10.
109. Elleuch, M.; Tahar, S. Formal Analysis of an IoT-Based Healthcare Application. In Proceedings of the 2023 IEEE Symposium on Computers and Communications (ISCC), Gammarth, Tunisia, 9–12 July 2023; pp. 1–5. [CrossRef]
110. Newaz, A.I.; Aris, A.; Sikder, A.K.; Uluagac, A.S. Systematic Threat Analysis of Modern Unified Healthcare Communication Systems. In Proceedings of the GLOBECOM-IEEE Global Communications Conference, Rio de Janeiro, Brazil, 4–8 December 2022; pp. 1404–1410. [CrossRef]
111. Arfi, F.; Courbis, A.L.; Lambolais, T.; Bughin, F.; Hayot, M. Formal verification of a telerehabilitation system through an abstraction and refinement approach using Uppaal. *IET Softw.* **2023**, *17*, 582–599. [CrossRef]
112. Nawaz, A.; Hasan, O.; Jabeen, S. Formal Verification of Deep Brain Stimulation Controllers for Parkinson's Disease Treatment. *Neural Comput.* **2023**, *35*, 671–698. [CrossRef] [PubMed]
113. Fernandes, H.R.; Gomes, G.F.; de Oliveira, A.C.P.; Campos, S.V.A. Stochastic Formal Model of PI3K/mTOR Pathway in Alzheimer's Disease for Drug Repurposing: An Evaluation of Rapamycin, LY294002, and NVP-BEZ235. *Sci. Comput. Program.* **2023**, *232*, 103028. [CrossRef]
114. Bilgram, A.; Jensen, P.G.; Jørgensen, K.Y.; Larsen, K.G.; Mikučionis, M.; Muñiz, M.; Poulsen, D.B.; Taankvist, P. An investigation of safe and near-optimal strategies for prevention of Covid-19 exposure using stochastic hybrid models and machine learning. *Decis. Anal. J.* **2022**, *5*, 100141. [CrossRef]
115. Cuartas, J.; Cortés, D.; Betancourt, J.S.; Aranda, J.; García, J.I.; Valencia, A.M.; Ortiz, J. Formal Verification of a Mechanical Ventilator using UPPAAL. In Proceedings of the International Workshop on Formal Techniques for Safety-Critical Systems, Caltais, Portugal, 22 October 2023.
116. Heuer, J.; Krenz-Bååth, R.; Obermaisser, R. Verifying Bio-Electronic Systems. In Proceedings of the 26th International Symposium on Design and Diagnostics of Electronic Circuits and Systems (DDECS), Tallinn, Estonia, 3–5 May 2023; IEEE: New York, NY, USA, 2023; pp. 161–166.
117. Novak, M.; Grobelna, I.; Nyman, U.; Szczesniak, P.; Blaabjerg, F. Statistical Performance Verification of the FS-MPC Algorithm Applied to the Matrix Converter. In Proceedings of the International Power Electronics Conference (IPEC-Himeji 2022-ECCE Asia), Himeji, Japan, 15–19 May 2022; IEEE: New York, NY, USA, 2022; pp. 76–82.

118. Mansour, A.N.A.; Grillo, S.; Ragaini, E.; Rossi, M. A Formal Approach to the Verification of Protection Systems in Low-Voltage Distribution Grids. In Proceedings of the IEEE/ACM 11th International Conference on Formal Methods in Software Engineering (FormaliSE), Melbourne, Australia, 14–15 May 2023; pp. 120–129. [CrossRef]
119. Wei, L.; Miao, W.; Zeng, Z. Collaborative Modeling Power Smart IoT Entity Services based on Extended Timed Automata. In Proceedings of the EMIE The 2nd International Conference on Electronic Materials and Information Engineering, Hangzhou, China, 15–17 April 2022; pp. 1–5.
120. Hmidi, Z.; Kahloul, L.; Benharzallah, S. A new Mobility and Energy Harvesting aware Medium Access Control (MEH-MAC) protocol: Modelling and performance evaluation. *Ad Hoc Netw.* **2023**, *142*, 103108. [CrossRef]
121. Kristjansen, M.; Kulkarni, A.; Jensen, P.G.; Teodorescu, R.; Larsen, K.G. Dual Balancing of SoC/SoT in Smart Batteries using Reinforcement Learning in Uppaal Stratego. In Proceedings of the Annual Conference of the IEEE Industrial Electronics Society (IECON), Singapore, 16–19 October 2023.
122. Soltani, R.; Volk, M.; Diamonte, L.; Lopuhaä-Zwakenberg, M.; Stoelinga, M. Optimal Spare Management via Statistical Model Checking: A Case Study in Research Reactors. In Proceedings of the International Conference on Formal Methods for Industrial Critical Systems, Antwerp, Belgium, 20–22 September 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 205–223.
123. Nagy, A.; Mansour, A.; Grillo, S.; Ragaini, E.; Rossi, M. Rigorous Automated Verification of Protection Systems in LV Distribution Grids. In Proceedings of the 2023 IEEE International Conference on Environment and Electrical Engineering and 2023 IEEE Industrial and Commercial Power Systems Europe (EEEIC/I&CPS Europe), Madrid, Spain, 6–9 June 2023; IEEE: New York, NY, USA, 2023; pp. 1–6.
124. Hansen, J.; Larsen, K.G.; Cuijpers, P.J. Balancing Flexible Production and Consumption of Energy using Resource Timed Automata. In Proceedings of the 11th Mediterranean Conference on Embedded Computing (MECO), Budva, Montenegro, 7–10 June 2022; IEEE: New York, NY, USA, 2022; pp. 1–6.
125. Novak, M.; Grobelna, I.; Nyman, U.M.; Szczesniak, P.; Blaabjerg, F. Modular Modeling and Statistical Validation for Grid-Connected FS-MPC-Controlled Matrix Converters. *IEEE Trans. Ind. Electron.* **2022**, *70*, 8613–8623. [CrossRef]
126. Thilakasiri, T.; Becker, M. An Exact Schedulability Analysis for Global Fixed-Priority Scheduling of the AER Task Model. In Proceedings of the 28th Asia and South Pacific Design Automation Conference, Tokyo, Japan, 16–19 January 2023; pp. 326–332.
127. Foughali, M.; Hladik, P.E.; Zuepke, A. Compositional Verification of Embedded Real-Time Systems. *J. Syst. Archit.* **2023**, *142*, 102928. [CrossRef]
128. Zavatteri, M.; Rizzi, R.; Villa, T. Dynamic controllability of temporal networks with instantaneous reaction. *Inf. Sci.* **2022**, *613*, 932–952. [CrossRef]
129. Xu, W.; Wu, X.; Zhao, Y.; Li, Y. Formal Verification and Analysis of Time-Sensitive Software-Defined Network Architecture. In Proceedings of the International Conference on Software Engineering and Knowledge Engineering, Virtual, 1–10 July 2022; pp. 369–375. Avaliable online: https://ksiresearchorg.ipage.com/seke/seke22paper/paper094.pdf (accessed on 28 May 2025).
130. An, D.; Pan, Z.; Gao, X.; Li, S.; Yin, L.; Li, T. stohMCharts: A Modeling Framework for Quantitative Performance Evaluation of Cyber-Physical-Social Systems. *IEEE Access* **2023**, *11*, 44660–44671. [CrossRef]
131. Basile, D.; Beek, M.H.t.; Lazreg, S.; Cordy, M.; Legay, A. Static detection of equivalent mutants in real-time model-based mutation testing: An Empirical Evaluation. *Empir. Softw. Eng.* **2022**, *27*, 160. [CrossRef]
132. Goorden, M.A.; Larsen, K.G.; Nielsen, J.E.; Nielsen, T.D.; Qian, W.; Rasmussen, M.R.; Zhao, G. Guaranteed safe controller synthesis for switched systems using analytical solutions. In Proceedings of the IEEE Conference on Control Technology and Applications (CCTA), Bridgetown, Barbados, 22 September 2023; pp. 784–790.
133. Cornanguer, L.; Largouët, C.; Rozé, L.; Termier, A. TAG: Learning timed automata from logs. In Proceedings of the AAAI Conference on Artificial Intelligence, Virtual, 22 February–1 March 2022; Volume 36, pp. 3949–3958. Avaliable online: https://ojs.aaai.org/index.php/AAAI/article/view/20311 (accessed on 28 May 2025).
134. Lestingi, L.; Sbrolli, C.; Scarmozzino, P.; Romeo, G.; Bersani, M.M.; Rossi, M. Formal modeling and verification of multi-robot interactive scenarios in service settings. In Proceedings of the IEEE/ACM 10th International Conference on Formal Methods in Software Engineering, Pittsburgh, PA, USA, 18–22 May 2022; pp. 80–90.
135. Lestingi, L.; Zerla, D.; Bersani, M.M.; Rossi, M. Specification, stochastic modeling and analysis of interactive service robotic applications. *Robot. Auton. Syst.* **2023**, *163*, 104387. [CrossRef]
136. Foughali, M.; Zuepke, A. Formal verification of real-time autonomous robots: An interdisciplinary approach. *Front. Robot. AI* **2022**, *9*, 791757. [CrossRef] [PubMed]
137. Bøgh, S.; Jensen, P.G.; Kristjansen, M.; Larsen, K.G.; Nyman, U. Distributed Fleet Management in Noisy Environments via Model-Predictive Control. In Proceedings of the International Conference on Automated Planning and Scheduling, Virtual, 13–24 June 2022; Volume 32, pp. 565–573. Avaliable online: https://ojs.aaai.org/index.php/ICAPS/article/view/19843 (accessed on 28 May 2025).
138. Praveen, A.T.; Gupta, A.; Bhattacharyya, S.; Muthalagu, R. Assuring Behavior of Multirobot Autonomous Systems With Translation From Formal Verification to ROS Simulation. *IEEE Syst. J.* **2022**, *16*, 5092–5100. [CrossRef]

139. Lestingi, L.; Manglaviti, A.; Marinaro, D.; Marinello, L.; Askarpour, M.; Bersani, M.M.; Rossi, M. Analyzing the impact of human errors on interactive service robotic scenarios via formal verification. *Softw. Syst. Model.* **2023**, 2, 473–502. [CrossRef]
140. Dust, L.; Gu, R.; Seceleanu, C.; Ekström, M.; Mubeen, S. Pattern-Based Verification of ROS 2 Nodes Using UPPAAL. In Proceedings of the International Conference on Formal Methods for Industrial Critical Systems, Antwerp, Belgium, 20–22 September 2023; pp. 57–75.
141. Lestingi, L.; Bersani, M.M.; Rossi, M. Model-Driven Development of Service Robot Applications Dealing With Uncertain Human Behavior. *IEEE Intell. Syst.* **2022**, *37*, 48–56. [CrossRef]
142. Bersani, M.M.; Camilli, M.; Lestingi, L.; Mirandola, R.; Rossi, M.; Scandurra, P. Architecting Explainable Service Robots. In Proceedings of the European Conference on Software Architecture, Istanbul, Turkey, 18–22 September 2023; pp. 153–169. [CrossRef]
143. Wang, W.; Schuppe, G.F.; Tumova, J. Decentralized Multi-agent Coordination under MITL Specifications and Communication Constraints. In Proceedings of the 31st Mediterranean Conference on Control and Automation (MED), Limassol, Cyprus, 26–29 June 2023; pp. 842–849.
144. Li, R.; Yin, J.; Zhu, H.; Vinh, P.C. Verification of rabbitmq with kerberos using timed automata. *Mob. Netw. Appl.* **2022**, *27*, 2049–2067. [CrossRef]
145. Wongsitthiphaithun, N.; Vatanawood, W. Transforming YAWL Workflows with Time Interval Constraints into Timed Automata. In Proceedings of the 19th International Joint Conference on Computer Science and Software Engineering (JCSSE), Bangkok, Thailand, 22–25 June 2022; pp. 1–6.
146. Saadi, A.; Hammal, Y.; Oussalah, M.C. Automata-Based Approach to Manage Self-Adaptive Component-Based Architectures. *Int. J. Softw. Innov. (IJSI)* **2022**, *10*, 1–22. [CrossRef]
147. Basile, D.; ter Beek, M.H. A runtime environment for contract automata. In Proceedings of the International Symposium on Formal Methods, Lübeck, Germany, 6–10 March 2023; pp. 550–567.
148. Han, D.; Cai, Y.; Chen, W.; Cui, Z.; Li, A. Timed-SAS: Modeling and Analyzing the Time Behaviors of Self-Adaptive Software under Uncertainty. *Appl. Sci.* **2023**, *13*, 2018. [CrossRef]
149. Atif, M.; Naseer, M.; Khan, A.S. Formal Analysis of Distributed Shared Memory Algorithms. *UMT Artif. Intell. Rev.* **2022**, *2*, 22–32. [CrossRef]
150. Silva, G.R.S.; Rodrigues, G.N.; Canedo, E.D. A Modeling Strategy for the Verification of Context-Oriented Chatbot Conversational Flows via Model Checking. *J. Univers. Comput. Sci.* **2023**, *29*, 805. [CrossRef]
151. Mishra, K.C.; Dutta, S. Colluder detection in SaaS cloud applications with subscription based license. *Multimed. Tools Appl.* **2023**, *82*, 12135–12149. [CrossRef]
152. Goorden, M.A.; Jensen, P.G.; Larsen, K.G.; Samusev, M.; Srba, J.; Zhao, G. STOMPC: Stochastic Model-Predictive Control with Uppaal Stratego. In Proceedings of the International Symposium on Automated Technology for Verification and Analysis, Virtual Event, 25–28 October 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 327–333.
153. Kim, E.H.; Larsen, K.G.; Goorden, M.A.; Nielsen, T.D. Controlling Stormwater Detention Ponds under Partial Observability. *J. Log. Algebr. Methods Program* **2024**, *141*, 100979. [CrossRef]
154. Weyns, D.; Gheibi, O.; Quin, F.; Van Der Donckt, J. Deep Learning for Effective and Efficient Reduction of Large Adaptation Spaces in Self-Adaptive Systems. *ACM Trans. Auton. Adapt. Syst.* **2022**, *17*, 1–12. [CrossRef]
155. Göttmann, H.; Caesar, B.; Beers, L.; Lochau, M.; Schürr, A.; Fay, A. Precomputing Reconfiguration Strategies Based on Stochastic Timed Game Automata. In Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems, Montreal, QC, Canada, 23–28 October 2022; MODELS '22; pp. 31–42. [CrossRef]
156. Weyns, D.; Iftikhar, U.M. ActivFORMS: A formally founded model-based approach to engineer self-adaptive systems. *ACM Trans. Softw. Eng. Methodol.* **2023**, *32*, 1–48. [CrossRef]
157. Fatima, K.; Sultana, S.; Abbasi, J.A.; Khalid, M.T. The Modeling and Verification of Trainify:(Tennis App). *Int. J. Emerg. Multidiscip. Comput. Sci. Artif. Intell.* **2022**, *1*, 26–34. [CrossRef]
158. Hasrat, I.R.; Jensen, P.G.; Larsen, K.G.; Srba, J. End-to-end heat-pump control using continuous time stochastic modelling and Uppaal Stratego. In Proceedings of the International Symposium on Theoretical Aspects of Software Engineering, Cluj-Napoca, Romania, 8–10 July 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 363–380.
159. Albano, M.; Cibin, N.; Golmohamadi, H.; Skou, A. Probabilistic Flexoffers in residential heat pumps considering uncertain weather forecast. *Energy Inform.* **2022**, *5*, 1–19. [CrossRef]
160. Hasrat, I.R.; Jensen, P.G.; Larsen, K.G.; Srba, J. A toolchain for domestic heat-pump control using Uppaal Stratego. *Sci. Comput. Program.* **2023**, *230*, 102987. [CrossRef]
161. Cibin, N.; Tibo, A.; Golmohamadi, H.; Skou, A.; Albano, M. Machine learning-based algorithms to estimate thermal dynamics of residential buildings with energy flexibility. *J. Build. Eng.* **2023**, *65*, 105683. [CrossRef]

162. Yang, Z.; Yuan, L.; Liu, Y. A scheme for train communication information management in onboard-centered train control system. In Proceedings of the International Conference on Frontiers of Traffic and Transportation Engineering (FTTE), Lanzhou, China, 17–19 June 2022; SPIE: Bellingham, WA, USA, 2022; Volume 12340, pp. 69–75.
163. Naumann, B.; Jakobs, C.; Werner, M. Formal analysis of timeliness in the RaSTA protocol. In Proceedings of the 17th Conference on Computer Science and Intelligence Systems (FedCSIS), Sofia, Bulgaria, 4–7 September 2022; IEEE: New York, NY, USA, 2022; pp. 505–514.
164. Sassi, I.; Ghazel, M.; El-Koursi, E.M. Statistical Model Checking for On-board Train Integrity Safety and Performance Analysis. In Proceedings of the European Conference on Safety and Reliability (ESREL), Dublin, Ireland, 28 August–1 September 2022; 8p.
165. Lukács, G.; Bartha, T. Practical UML subset for railway engineers to support formal modeling. *Trans. Motauto World* **2022**, *7*, 56–59.
166. Niu, R.; You, S. Research on run-time risk evaluation method based on operating scenario data for autonomous train. *Accid. Anal. Prev.* **2022**, *178*, 106855. [CrossRef]
167. Lin, J.; Min, X. Quantitative safety analysis of train control system based on statistical model checking. *Arch. Transp.* **2022**, *61*.
168. Kochan, A.; Daszczuk, W.B.; Grabski, W.; Karolak, J. Formal Verification of the European Train Control System (ETCS) for Better Energy Efficiency Using a Timed and Asynchronous Model. *Energies* **2023**, *16*, 3602. [CrossRef]
169. Basile, D.; ter Beek, M.H.; Ferrari, A.; Legay, A. Exploring the ERTMS/ETCS full moving block specification: An experience with formal methods. *Int. J. Softw. Tools Technol. Transf.* **2022**, *24*, 351–370. [CrossRef]
170. Lukács, G.; Bartha, T. Formal modeling and verification of the functionality of electronic urban railway control systems through a case study. *Urban Rail Transit* **2022**, *8*, 217–245. [CrossRef]
171. Himrane, O.; Beugin, J.; Ghazel, M. Implementation of a Model-Oriented Approach for Supporting Safe Integration of GNSS-Based Virtual Balises in ERTMS/ETCS Level 3. *IEEE Open J. Intell. Transp. Syst.* **2023**, *4*, 294–310. [CrossRef]
172. Lin, J.; Min, X.; Chai, J. Model-Based Safety Analysis of Movement Authority Scenario in TcCBTC system. In *Proceedings of the Journal of Physics: Conference Series*; IOP Publishing: Bristol, UK, 2022; Volume 2246, p. 012077.
173. Saddem-Yagoubi, R.; Sanwal, M.U.; Libutti, S.; Benerecetti, M.; Beugin, J.; Flammini, F.; Ghazel, M.; Janssen, B.; Marrone, S.; Mogavero, F.; et al. Toward Usable Formal Models for Safety and Performance Evaluation of ERTMS/ETCS Level 3: The PERFORMINGRAIL Project. In Proceedings of the 32nd European Safety and Reliability Conference, Dublin, Ireland, 28 August–1 September 2022; 8p. Available online: https://swepub.kb.se/bib/swepub:oai:DiVA.org:mdh-69010?tab2=abs&language=en (accessed on 28 May 2025).
174. Wang, Z.; Liu, J.; Yi, L.; Wang, G. Fire Linkage Scheme Design and Modeling Verification of Urban Rail Transit. In Proceedings of the 5th International Conference on Artificial Intelligence and Big Data (ICAIBD), Chengdu, China, 27–30 May 2022; pp. 648–652. [CrossRef]
175. Proença, J.; Borrami, S.; Sanchez de Nova, J.; Pereira, D.; Nandi, G.S. Verification of multiple models of a safety-critical motor controller in railway systems. In Proceedings of the International Conference on Reliability, Safety, and Security of Railway Systems, Paris, France, 1 June 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 83–94.
176. Seisenberger, M.; ter Beek, M.H.; Fan, X.; Ferrari, A.; Haxthausen, A.E.; James, P.; Lawrence, A.; Luttik, B.; van de Pol, J.; Wimmer, S. Safe and secure future AI-driven railway technologies: Challenges for formal methods in railway. In Proceedings of the International Symposium on Leveraging Applications of Formal Methods, Rhodes, Greece, 22–30 October 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 246–268.
177. Fantechi, A.; Gnesi, S.; Gori, G. Future train control systems: Challenges for dependability assessment. In Proceedings of the International Symposium on Leveraging Applications of Formal Methods, Rhodes, Greece, 22–30 October 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 269–285.
178. Kobialka, P.; Tapia Tarifa, S.L.; Bergersen, G.R.; Johnsen, E.B. Weighted games for user journeys. In Proceedings of the International Conference on Software Engineering and Formal Methods, Aveiro, Portugal, 26–30 September 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 253–270.
179. Kobialka, P.; Mannhardt, F.; Tapia Tarifa, S.L.; Johnsen, E.B. Building User Journey Games from Multi-party Event Logs. In Proceedings of the International Conference on Process Mining, Bolzano, Italy, 3 November 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 71–83.
180. Yasmine, A.; Ameur-Boulifa, R.; Guitton-Ouhamou, P.; Pacalet, R. Automatic Support for Requirements Validation. In Proceedings of the 11th Embedded Real-Time Systems Congress (ERTS'2022), Toulouse, France, 1–2 June 2022. Available online: https://telecom-paris.hal.science/hal-03689243v1/document (accessed on 28 May 2025).
181. Bendík, J.; Sencan, A.; Gol, E.A.; Černá, I. Timed automata robustness analysis via model checking. *Log. Methods Comput. Sci.* **2022**, *18*. [CrossRef]
182. Alam, M.T.; Halder, R.; Maiti, A. Formal Verification of Pub-Sub Blockchain Interoperability Protocol using Stochastic Timed Automata. *Front. Blockchain* **2023**, *6*, 1248962. [CrossRef]

183. Bouyer, P.; Gastin, P.; Herbreteau, F.; Sankur, O.; Srivathsan, B. Zone-based verification of timed automata: Extrapolations, simulations and what next? In Proceedings of the International Conference on Formal Modeling and Analysis of Timed Systems, Warsaw, Poland, 13–15 September 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 16–42.
184. Cortellessa, V.; Pomante, L.; Stoico, V. From UML/MARTE Specifications to ESL HW/SW Co-Design: Early Functional Verification and Timing Validation. In Proceedings of the Companion of the ACM/SPEC International Conference on Performance Engineering, Coimbra, Portugal, 15–19 April 2023; pp. 373–380.
185. Guin, J.; Vain, J.; Tsiopoulos, L.; Valdek, G. Temporal Multi-View Contracts Help Developing Efficient Test Models. *Balt. J. Mod. Comput.* **2022**, *10*, 710–737. Available online: https://www.bjmc.lu.lv/fileadmin/user_upload/lu_portal/projekti/bjmc/Contents/10_4_07_Guin.pdf (accessed on 28 May 2025). [CrossRef]
186. Mahajan, A.; Martin, S.; Watt, S.J.; Wong, M.W.H.M. Compliance through model checking. In Proceedings of the International Workshop on AI Compliance Mechanism WAICOM, Saarbrücken, Germany, 14 December 2022. Available online: https://ink.library.smu.edu.sg/cclaw/3/ (accessed on 28 May 2025).
187. Tiwari, S.; Iyer, K.; Enoiu, E.P. Combining Model-Based Testing and Automated Analysis of Behavioural Models using GraphWalker and UPPAAL. In Proceedings of the 29th Asia-Pacific Software Engineering Conference (APSEC), Tokyo, Japan, 6–9 December 2022; IEEE: New York, NY, USA, 2022; pp. 452–456. Available online: https://ieeexplore.ieee.org/abstract/document/10043283 (accessed on 28 May 2025).
188. Lehmann, S.; Schupp, S. Bounded DBM-based clock state construction for timed automata in Uppaal. *Int. J. Softw. Tools Technol. Transf.* **2023**, *25*, 19–47. [CrossRef]
189. Chen, H.; Su, Y.; Zhang, M.; Liu, Z.; Mi, J. Learning Assumptions for Compositional Verification of Timed Automata. In Proceedings of the International Conference on Computer Aided Verification, Paris, France, 17–22 July 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 40–61.
190. Jensen, P.G.; Kiviriga, A.; Guldstrand Larsen, K.; Nyman, U.; Mijačika, A.; Høiriis Mortensen, J. Monte Carlo Tree Search for Priced Timed Automata. In Proceedings of the International Conference on Quantitative Evaluation of Systems, Warsaw, Poland, 13–16 September 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 381–398.
191. Larsen, K.G.; Legay, A.; Mikučionis, M.; Poulsen, D.B. Importance splitting in uppaal. In Proceedings of the International Symposium on Leveraging Applications of Formal Methods, Rhodes, Greece, 22–30 October 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 433–447.
192. Vogel, T.; Carwehl, M.; Rodrigues, G.N.; Grunske, L. A property specification pattern catalog for real-time system verification with UPPAAL. *Inf. Softw. Technol.* **2023**, *154*, 107100. [CrossRef]
193. Vain, J.; Tsiopoulos, L.; Kanter, G. Provably correct aspect-oriented modeling with UPPAAL timed automata. In *System Assurances*; Elsevier: Amsterdam, The Netherlands, 2022; pp. 447–476.
194. Guin, J.; Vain, J.; Tsiopoulos, L.; Valdek, G. Temporal multi-view contracts for efficient test models. In Proceedings of the International Baltic Conference on Digital Business and Intelligent Systems, Vilnius, Lithuania, 30 June 2024; Springer: Berlin/Heidelberg, Germany, 2022; pp. 136–151.
195. Johri, P.; Anand, A.; Vain, J.; Singh, J.; Quasim, M.T. *System Assurances: Modeling and Management*; Academic Press: Cambridge, MA, USA, 2022.
196. Tigane, S.; Guerrouf, F.; Hamani, N.; Kahloul, L.; Khalgui, M.; Ali, M.A. Dynamic timed automata for reconfigurable system modeling and verification. *Axioms* **2023**, *12*, 230. [CrossRef]
197. Melchert, S.B.J. Spreadsheet-based Configuration of Families of Real-Time Specifications. *TiCSA* 2023, *submitted to workshop.* Available online: https://arxiv.org/abs/2310.20395 (accessed on 28 May 2025).
198. Cicirelli, F.; Nigro, L. Analyzing stochastic reward nets by model checking and parallel simulation. *Simul. Model. Pract. Theory* **2022**, *116*, 102467. [CrossRef]
199. Peres, F.; Ghazel, M. A proven translation from a UML state machine subset to timed automata. *ACM Trans. Embed. Comput. Syst.* **2023**, *23*, 1–33. [CrossRef]
200. Zaman, M.; Atif, M.; Naseer, M. Formal Verification of Twin Clutch Gear Control. *VAWKUM Trans. Comput. Sci.* **2022**, *10*, 24–33. [CrossRef]
201. Kitahara, Y.; Nakamura, M.; Sakakibara, K. An Investigation of Formal Verification of Control Policy of Multi-Car Elevator Systems Using Statistical Model Checking. In Proceedings of the International Conference on Machine Learning and Cybernetics (ICMLC), Tokyo, Japan, 9–11 September 2022; pp. 118–193. Available online: https://ieeexplore.ieee.org/document/9941319 (accessed on 28 May 2025).
202. Palliwar, A.; Pinisetty, S. Using gossip enabled distributed circuit breaking for improving resiliency of distributed systems. In Proceedings of the IEEE 19th International Conference on Software Architecture (ICSA), Honolulu, HI, USA, 12–15 March 2022; IEEE: New York, NY, USA, 2022; pp. 13–23.

203. Palliwar, A.; Pinisetty, S. Artifact for Measuring the Relative Efficacy of Gossip Enabled Distributed Circuit Breaking. In Proceedings of the 2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C), Honolulu, HI, USA, 12–15 March 2022; p. 55. [CrossRef]
204. Daszczuk, W.B. Modeling and Verification of Asynchronous Systems Using Timed Integrated Model of Distributed Systems. *Sensors* **2022**, *22*, 1157. [CrossRef]
205. Li, S.; Yang, Q.; Xing, J.; Chen, W.; Zou, R. A Foundation Model for Building Digital Twins: A Case Study of a Chiller. *Buildings* **2022**, *12*, 1079. [CrossRef]
206. Zhang, M.; Teng, Y.; Kong, H.; Baugh, J.; Su, Y.; Mi, J.; Du, B. Automatic modelling and verification of Autosar architectures. *J. Syst. Softw.* **2023**, *201*, 111675. [CrossRef]
207. Asokan, S.; Kochaleema, K.; Kumar, G.S. Formal Modelling and Verification of the Clock Synchronisation Algorithm of FlexRay. *Def. Sci. J.* **2023**, *73*, 41–50. [CrossRef]
208. Bersani, M.M.; Camilli, M.; Lestingi, L.; Mirandola, R.; Rossi, M. Explainable human-machine teaming using model checking and interpretable machine learning. In Proceedings of the IEEE/ACM 11th International Conference on Formal Methods in Software Engineering (FormaliSE), Melbourne, Australia, 14–15 May 2023; IEEE: New York, NY, USA, 2023; pp. 18–28.
209. Wang, X.; Guo, Y.; Lu, N.; He, P. UAV Cluster Behavior Modeling Based on Spatial-Temporal Hybrid Petri Net. *Appl. Sci.* **2023**, *13*, 762. [CrossRef]
210. Cuijpers, P.J.; Hansen, J.; Larsen, K.G. Assume-Guarantee Reasoning for Additive Hybrid Behaviour. In *Theories of Programming and Formal Methods: Essays Dedicated to Jifeng He on the Occasion of His 80th Birthday*; Springer: Berlin/Heidelberg, Germany, 2023; pp. 297–322. [CrossRef]
211. Batteux, M.; Prosvirnova, T.; Rauzy, A. A guided tour of AltaRica wizard, the AltaRica 3.0 integrated modeling environment. In Proceedings of the 32nd European Safety and Reliability Conference (ESREL 2022), Dublin, Ireland, 28 August–1 September 2022; pp. 2246–2253. Available online: https://www.rpsonline.com.sg/proceedings/esrel2022/html/copy.html (accessed on 28 May 2025).
212. André, É.; Liu, S.; Liu, Y.; Choppy, C.; Sun, J.; Dong, J.S. Formalizing UML state machines for automated verification–A survey. *ACM Comput. Surv.* **2023**, *55*, 1–47. [CrossRef]